

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Extensión e
implantación de
Solución Informática
para el aprendizaje
del lenguaje musical

Curso 2025/26

Alumno/a:

José Luis Garrido Castro

Director/es:

Javier Criado Rodríguez

AGRADECIMIENTOS

Me gustaría dar las gracias, en primer lugar, a la Escuela de Música de Huércal de Almería, al Real Conservatorio Profesional de Música de Almería y la Escuela de Música Manuel Sánchez Alonso de Canjáyar por formarme durante mis 17 últimos años, poner en mi vida este maravilloso arte que es la música y por prestarme su valiosa ayuda para este proyecto.

En segundo lugar, a mi familia ya que siempre me han apoyado a lo largo de toda mi vida y se que se sienten muy orgullosos de mis logros, en especial, los conseguidos a lo largo de este grado de Ingeniería Informática. Me han dado todo y no me cabe duda que siempre me lo darán.

Después, cabe mencionar a mis amigos que siempre estaban ahí para animarme en momentos malos y sacarme de casa cuando me hacía falta.

En particular, quería destacar a mi compañero Jesús, con el cual llevo coincidiendo desde el instituto, manteniendo una estrecha amistad que se ha fortalecido durante el grado pasando grandes momentos y ayudándonos de forma recíproca en lo que hemos necesitado cada uno.

A mi pareja que hasta hoy no ha quedado día que no me haya preguntado como me iba con todos mis proyectos (como este) y que siempre consigue mandarme un empujón y sacarme una sonrisa para seguir llevando todo a cabo.

Por último, a todos los profesores que me han ayudado a lo largo de todo el grado y que me han transmitido todos los conocimientos posibles así como permitirme desarrollar la gran cantidad de nuevas habilidades que he conseguido durante estos años.

ÍNDICE GENERAL

	Página
1. Contexto del proyecto y punto de partida	1
1.1. Marco teórico, investigación y estudio previo	1
1.2. Resultados del proyecto base y líneas de evolución	1
2. Objetivos del proyecto	3
3. Definición de la solución extendida	4
3.1. Análisis de la solución base y motivación de la extensión	4
3.2. Especificación formal de la solución extendida	5
3.3. Objetivos funcionales del sistema extendido	7
3.4. Diagrama UML de casos de uso del sistema extendido	7
3.5. Definición del diagrama de casos de uso	8
3.5.1. Casos de uso nuevos o modificados	9
3.5.1.1. Caso de uso: Importar usuarios y grupos desde archivo formateado	9
3.5.1.2. Caso de uso que sustituye Crear profesor: Gestionar profesores	9
3.5.1.3. Caso de uso que sustituye Crear alumno: Gestionar alumnos	9
3.5.1.4. Caso de uso existente ampliado: Gestionar grupos	9
3.5.1.5. Caso de uso existente ampliado: Interactuar con cuestionarios	10
3.5.2. Requisitos funcionales nuevos	10
3.6. Requisitos no funcionales nuevos	14
3.7. Tecnologías y arquitectura de la solución extendida	15
3.7.1. Pila tecnológica heredada del proyecto base	15
3.7.2. Tecnologías incorporadas en la solución extendida	16
3.7.2.1. Servicio de inferencia externa mediante Groq	16
3.7.2.2. Capacitor como evolución del soporte móvil híbrido	17
3.7.2.3. Android Studio y entorno de construcción móvil	18
4. Planificación del proyecto	20
5. Desarrollo de la aplicación backend	23

5.1. Modelado de datos	23
5.2. Implementación	24
5.2.1. Operaciones lógicas (revisión y eliminación)	25
5.2.2. Importación masiva desde CSV (normalización, validación y creación)	25
5.2.3. Servicio de IA con Groq (integración por API y capa de servicios)	27
5.3. Testing	29
6. Desarrollo de la aplicación frontend	32
6.1. Implementación	32
6.1.1. Exportación a Android con Capacitor	32
6.1.1.1. Ajuste de menús del sistema	32
6.1.1.2. Apertura de PDFs en Android (Filesystem + FileOpener)	33
6.1.1.3. Estilos específicos para móvil	35
6.1.1.4. Ajustes finales en Android Studio (Iconos, Pantallas de carga y XML)	35
6.2. Diseño y experiencia de usuario (UX/UI)	36
6.2.1. Continuidad visual y coherencia de interacción	36
6.2.2. Efectos visibles de la adaptación a móvil	36
6.2.3. Ejemplos visuales de la solución extendida	37
6.3. Testing	39
7. Despliegue del sistema	42
7.1. Despliegue web	42
7.2. Despliegue móvil: generación de la aplicación Android	42
7.2.1. Generación del contenedor Android mediante Capacitor	43
7.2.2. Construcción y validación en Android Studio	43
7.2.3. Generación del paquete instalable	43
7.3. Valor añadido del despliegue móvil	44
8. Plan de implantación	45
8.1. Objetivos del plan de implantación	45
8.2. Contexto de implantación en centros educativos	46
8.3. Estrategia de implantación progresiva	46
8.3.1. Fase 1: Despliegue piloto	47
8.3.2. Fase 2: Despliegue parcial	48
8.3.3. Fase 3: Despliegue completo	48
8.4. Migración tecnológica hacia un entorno productivo	49

8.4.1.	Justificación de la migración	49
8.4.2.	Migración del backend y frontend a Microsoft Azure	49
8.4.2.1.	Azure App Service como plataforma de ejecución	50
8.4.2.2.	Arquitectura de despliegue propuesta	50
8.4.2.3.	Proceso de migración	51
8.4.2.4.	Pipeline de despliegue automatizado	51
8.4.3.	Escalado de la base de datos con MongoDB Atlas	55
8.4.3.1.	Arquitectura de clúster y escalabilidad	55
8.4.3.2.	Plan de escalado progresivo	55
8.4.3.3.	Integración con la arquitectura global	55
8.5.	Publicación y distribución oficial de la aplicación móvil	56
8.5.1.	Justificación de la publicación en tienda oficial	56
8.5.2.	Preparación técnica para la publicación	56
8.5.3.	Generación del paquete Android App Bundle (AAB)	57
8.5.4.	Proceso de publicación en Google Play Console	57
8.5.5.	Integración en la estrategia de implantación	57
8.6.	Migración del sistema de almacenamiento de archivos	58
8.7.	Gestión profesional del correo electrónico	59
8.8.	Seguridad, protección de datos y cumplimiento normativo	60
8.8.1.	Plan operativo de seguridad	60
8.8.2.	Protección de datos personales y marco normativo	60
8.8.3.	Gestión de credenciales y control de accesos	61
8.8.4.	Integración de servicios externos (correo, contraseñas, IA)	61
8.8.5.	Protección frente a pérdida de información (backups)	61
8.8.6.	Seguridad en entorno móvil	61
8.9.	Formación de usuarios y adopción del sistema	62
8.9.1.	Formación del profesorado	62
8.9.2.	Formación del alumnado	62
8.9.3.	Documentación y recursos de apoyo	62
8.10.	Cronograma de implantación	63
8.11.	Validación práctica en entorno educativo real	64
8.11.1.	Contexto de la demostración	65
8.11.2.	Valoración del profesorado	65
8.11.3.	Reacciones del alumnado	66
8.11.4.	Problemas detectados y mejoras aplicadas	66

8.11.5. Valor académico de la validación empírica	67
8.12. Síntesis del plan de implantación	68
9. Conclusiones y líneas de trabajo futuro	69
9.1. Conclusiones	69
9.2. Líneas de trabajo futuro	70
9.2.1. Ejecución completa del plan de implantación en un centro educativo	70
9.2.2. Extensión continua de funcionalidades e innovación pedagógica	70
9.2.3. Apertura del sistema a usuarios externos y acceso sin autenticación	71
9.2.4. Tecnologías de mantenimiento, monitorización y gestión de logs	71
9.2.5. Refuerzo de la seguridad y mejora en la gestión de credenciales	71

ÍNDICE DE FIGURAS

3.1. Diagrama UML de casos de uso del sistema extendido	8
3.2. Logo Groq	16
3.3. Logo Capacitor	18
3.4. Logo Android Studio.	18
3.5. Arquitectura global de la aplicación (sin cambios respecto al proyecto base)	19
4.1. Planificación del proyecto con nuevas tareas destacadas	21
4.2. Planificación de las tareas principales en horas	22
5.1. Modelo conceptual de datos del sistema	24
6.1. Menú doble (lateral e inferior)	37
6.2. Vista de la rama de ritmo en versión web (izquierda) y Android (derecha).	38
6.3. Vista de cuestionario en versión web (izquierda) y Android (derecha).	38
6.4. Icono desde vista nativa	38
7.1. Aplicación instalada en un dispositivo Android	44
8.1. Logo Microsoft Azure	54
8.2. Logo Firebase Storage	59
8.3. Cronograma orientativo de implantación	64
8.4. Demostración de la aplicación SolfEdge en un entorno real de aula	67

ÍNDICE DE TABLAS

3.1. Caso de uso: Importar usuarios y grupos desde archivo formateado	9
3.2. Caso de uso: Gestionar profesores	9
3.3. Caso de uso: Gestionar alumnos	9
3.4. Caso de uso: Gestionar grupos	9
3.5. Caso de uso: Interactuar con cuestionarios	10
3.6. Requisito funcional: RF-31 Importar usuarios y grupos desde archivo formateado	10
3.7. Requisito funcional: RF-32 Revisar profesores	11
3.8. Requisito funcional: RF-33 Eliminar profesor	11
3.9. Requisito funcional: RF-34 Revisar alumnos	12
3.10. Requisito funcional: RF-35 Eliminar alumno	12
3.11. Requisito funcional: RF-36 Eliminar grupo	13
3.12. Requisito funcional: RF-37 Solicitar pista durante un cuestionario	14
3.13. Requisito no funcional: RNF-08 Portabilidad a Android	14

1 CONTEXTO DEL PROYECTO Y PUNTO DE PARTIDA

El presente proyecto se concibe como una extensión y evolución directa del Trabajo Fin de Grado desarrollado previamente, en el que se abordó el diseño, desarrollo y validación de una solución informática orientada a la enseñanza del lenguaje musical mediante una aplicación web con arquitectura cliente-servidor.

1.1 MARCO TEÓRICO, INVESTIGACIÓN Y ESTUDIO PREVIO

En el proyecto original se estableció, en primer lugar, un marco teórico que permitió contextualizar el lenguaje musical como disciplina educativa, analizando su papel dentro de la formación musical y las dificultades habituales asociadas a su aprendizaje y enseñanza. Este marco teórico puso de manifiesto una serie de problemáticas recurrentes, tales como la falta de recursos digitales específicos, la escasa adaptación de las herramientas existentes a las dinámicas reales del aula y la dificultad para realizar un seguimiento individualizado del alumnado en este ámbito.

A partir de esta base teórica, se llevó a cabo una investigación empírica apoyada en encuestas y entrevistas a profesores y alumnos de lenguaje musical, cuyo objetivo fue contrastar las problemáticas identificadas y detectar necesidades concretas desde la experiencia directa en entornos educativos reales. Paralelamente, se realizó un estudio basado en una revisión bibliográfica de trabajos previos y soluciones existentes, lo que permitió situar el proyecto dentro del estado del arte y analizar las limitaciones y oportunidades de las herramientas disponibles en la actualidad.

La combinación de marco teórico, investigación empírica y revisión bibliográfica proporcionó una base sólida para la definición de los requisitos funcionales y no funcionales del sistema, así como para la formulación de los objetivos del proyecto. Como resultado de este proceso, se diseñó e implementó una plataforma informática compuesta por un backend basado en una API REST y un frontend híbrido accesible desde distintos dispositivos.

1.2 RESULTADOS DEL PROYECTO BASE Y LÍNEAS DE EVOLUCIÓN

La aplicación desarrollada en la versión inicial permite gestionar usuarios, grupos, material de clase, calificaciones, cuestionarios, tareas y comunicaciones internas, incorporando además mecanismos de autenticación, pruebas automatizadas, documentación técnica y procesos de despliegue continuo. Esta primera versión del sistema fue validada funcionalmente, demostrando su viabilidad técnica y su adecuación inicial a las necesidades detectadas.



No obstante, tanto las conclusiones del proyecto como la experiencia práctica derivada del uso de la aplicación evidenciaron que la solución implementada, aun siendo plenamente funcional, presenta margen de mejora y evolución. En particular, se identificaron oportunidades para refinar determinadas funcionalidades existentes, incorporar nuevas capacidades que amplíen el alcance de la aplicación y abordar de forma más profunda su implantación en entornos educativos reales. Estas líneas de evolución quedaron recogidas explícitamente como trabajo futuro en el proyecto original.

En este contexto, el presente proyecto no pretende repetir ni reformular el marco teórico ni los estudios ya realizados, sino apoyarse en ellos como base validada para dar un paso adicional. El foco se desplaza desde el desarrollo inicial de la solución hacia su extensión funcional y su preparación para una implantación realista y sostenible, considerando aspectos técnicos, organizativos y de seguridad que resultan fundamentales en un escenario de uso continuado.

De este modo, este trabajo se plantea como una continuación natural y coherente del proyecto previo, orientada a consolidar la solución desarrollada y a sentar las bases necesarias para su evolución e implantación efectiva en centros educativos, manteniendo la alineación con la problemática detectada y con los principios pedagógicos que motivaron el trabajo previo.

2 OBJETIVOS DEL PROYECTO

El objetivo general de este proyecto consiste en extender e implantar la solución informática desarrollada en el Trabajo Fin de Grado previo, llevándola a un nivel superior tanto en términos funcionales como de preparación para su uso en entornos educativos reales.

A partir de este objetivo general, se definen los siguientes subobjetivos específicos:

- **Mejorar y refinar funcionalidades ya existentes en la aplicación**, optimizando su usabilidad y adecuación a las necesidades detectadas en el contexto educativo.
- **Diseñar e incorporar nuevas funcionalidades innovadoras** que amplíen el alcance de la aplicación y aporten un valor añadido respecto a la versión inicial.
- **Aprovechar el diseño responsive y la arquitectura híbrida de la aplicación para facilitar su adaptación y despliegue en sistemas móviles**, permitiendo su ejecución en dispositivos Android y iOS sin necesidad de desarrollar una base de código independiente, y garantizando una experiencia de usuario coherente y optimizada en distintos tamaños de pantalla.
- **Definir un plan de implantación detallado** que contemple la adaptación tecnológica del sistema, la mejora de aspectos de seguridad, la gestión de archivos y datos, así como la viabilidad de su despliegue en centros educativos reales.

Estos objetivos se plantean manteniendo la coherencia con el proyecto base y aprovechando las conclusiones extraídas del proceso de investigación y desarrollo previo.

3 DEFINICIÓN DE LA SOLUCIÓN EXTENDIDA

Este complemento parte de la solución desarrollada en versión base y se apoya en su definición previa. La aplicación original resolvía los procesos esenciales de gestión académica y comunicación asociados a la enseñanza del lenguaje musical, ofreciendo una plataforma con roles diferenciados (administrador, profesor y alumno), ramas de contenido, tareas y cuestionarios, sistema de mensajería y calificaciones.

Sin embargo, el propio enfoque del proyecto —y especialmente sus conclusiones y líneas de trabajo futuro— asumía que una primera versión funcional no agota el problema: al llevar la solución a un entorno real aparecen necesidades operativas y oportunidades de mejora que no siempre resultan evidentes durante un desarrollo estrictamente académico. En particular, el uso prolongado en contextos educativos reales tiende a revelar fricciones relacionadas con la administración masiva, la gestión completa del ciclo de vida de usuarios y grupos y la experiencia de uso del alumnado en actividades evaluables.

En este punto, el objetivo de este complemento no es replantear la solución desde cero, sino evolucionarla de manera controlada. Se mantiene la arquitectura general, la filosofía de funcionamiento y la mayor parte de requisitos ya definidos en el documento principal, lo que se incorpora ahora se centra en:

- **Operatividad y administración real:** Reducir carga manual y facilitar tareas frecuentes en la práctica (altas masivas, etc.).
- **Gestión completa del ciclo de vida:** Incorporar operaciones de revisión y eliminación con comportamiento coherente y seguro.
- **Mejora de la experiencia del alumno:** Introducir mecanismos de apoyo durante el trabajo con cuestionarios para favorecer el aprendizaje sin romper la coherencia pedagógica.

Como en la definición de la solución base, el proceso se plantea como un avance gradual desde un lenguaje natural todavía interpretativo hasta una definición progresivamente más objetiva. Primero se actualiza la especificación formal con las nuevas capacidades, después se refleja la ampliación en el diagrama de casos de uso, por último, se describen en detalle únicamente los casos de uso y requisitos afectados por los cambios (sin repetir lo ya definido previamente).

3.1 ANÁLISIS DE LA SOLUCIÓN BASE Y MOTIVACIÓN DE LA EXTENSIÓN

La solución original cubría de forma adecuada la operativa cotidiana de aula: creación de usuarios, creación y selección de grupos, publicación de material y tareas, comunicación con el alumnado, cues-

tionarios en la rama de teoría y gestión de calificaciones. Este conjunto de funcionalidades constituye una base sólida y coherente con la estructura pedagógica del lenguaje musical.

No obstante, al considerar una implantación real aparecen situaciones que requieren una respuesta más específica. Por ejemplo, la creación individual de usuarios funciona correctamente en grupos pequeños, pero se vuelve costosa cuando el número de alumnos aumenta o cuando se gestionan varios grupos simultáneos (escuelas con múltiples niveles, grupos por curso, etc.). De forma similar, en un entorno real es habitual que existan bajas, cambios de grupo, reorganizaciones o la necesidad de eliminar grupos completos cuando finaliza un curso o cuando se detectan altas erróneas, y estas operaciones deben dejar el sistema en un estado consistente.

Por último, en relación con el aprendizaje, la realización de cuestionarios tipo test (especialmente en teoría) tiende a producir bloqueos cuando el alumno no recuerda un concepto puntual. En una sesión real, el profesor suele ofrecer una pista o guía mínima para reconducir el razonamiento sin revelar la respuesta. Replicar parte de esa asistencia dentro de la herramienta puede mejorar la experiencia del alumno y reducir frustración, siempre que se haga con límites y con una intención pedagógica clara.

A partir de este análisis, se definen nuevas extensiones funcionales sobre la base ya existente.

3.2 ESPECIFICACIÓN FORMAL DE LA SOLUCIÓN EXTENDIDA

En esta sección se conserva la especificación formal descrita en el proyecto original, incorporando únicamente las ampliaciones necesarias para reflejar la evolución del sistema. El objetivo es mantener continuidad, evitar redundancias y, al mismo tiempo, dejar completamente claro qué aporta este complemento. Las partes incorporadas a la especificación en esta fase estarán destacadas en color rojo.

El sistema permitirá el acceso mediante un mecanismo de inicio de sesión en el que podrán autenticarse tres tipos de usuarios: administrador, profesor y alumno. Cada usuario tendrá acceso únicamente a las funcionalidades correspondientes a su rol, sin posibilidad de utilizar las funciones asignadas a otros.

No existirá un proceso de registro. El administrador será creado de forma automática y, una vez dentro de la aplicación, tendrá la capacidad de crear las cuentas de los profesores manualmente. Para ello, deberá introducir los siguientes datos: nombre, primer apellido, segundo apellido y dirección de correo electrónico. Se realizará una comprobación de validez de los datos introducidos (formato correcto del nombre, de los apellidos y del correo electrónico) y una verificación de unicidad del correo electrónico en la base de datos.

En el proceso de creación, el sistema generará automáticamente un nombre de usuario a partir de la primera letra del nombre y de los apellidos. Por ejemplo, para un usuario denominado José Luis Garrido Castro, el identificador generado será jgc. En caso de que existan coincidencias con identificadores ya presentes en la base de datos, se añadirá un número consecutivo equivalente al total de usuarios con la misma raíz más uno, de modo que si ya existen 40 usuarios con identificadores iniciados en jgc, el nuevo identificador será jgc41. La contraseña será generada automáticamente mediante el uso de un servicio externo de generación de contraseñas seguras. Una vez creadas las credenciales, estas serán enviadas por correo electrónico al profesor.

Del mismo modo, el administrador podrá revisar la lista de profesores creados y eliminarlos. La eliminación de un profesor implicará la eliminación de toda la información asociada creada bajo su responsabilidad (material, tareas, cuestionarios, grupos y alumnado asociado a dichos grupos).

Además, el sistema permitirá al administrador realizar la creación de usuarios y grupos de manera automatizada. El fichero contendrá la información necesaria para dar de alta profesores y/o alumnos y para asociarlos a grupos, aplicando las mismas validaciones de datos y unicidad ya establecidas para la creación manual.

El profesor, a su vez, tendrá la capacidad de crear cuentas de alumnos mediante un procedimiento equivalente al descrito para la creación de profesores manual, **revisar y eliminar a sus alumnos (eliminando así sus registros de entregas y calificaciones)**.

Además, el profesor podrá crear grupos de clase, añadir alumnos a los mismos **y eliminar cualquiera de los grupos que haya creado borrando también de forma sistemática el material que tenga asociado**. Un alumno podrá pertenecer a varios grupos de forma simultánea.

También podrá enviar mensajes a unos alumnos o grupos completos determinados y podrá adjuntar las calificaciones generales del curso seleccionando a un alumno en concreto.

El sistema dispondrá de un área denominada administración en la que estarán disponibles las operaciones mencionadas anteriormente.

Fuera de esta sección, la aplicación contará con cuatro apartados denominados **Ritmo, Entonación, Audición y Teoría**. Cada apartado estará compuesto por dos secciones diferenciadas:

1. **Libro del curso:** Permitirá al profesor subir un único archivo en formato PDF correspondiente al grupo seleccionado. Si se sube un nuevo archivo, el anterior será sustituido. El profesor podrá eliminar o reemplazar el archivo en cualquier momento.
2. **Tareas a realizar:** Permitirá al profesor crear tareas asociadas a un grupo determinado. Cada tarea constará de una descripción y, opcionalmente, de un material de apoyo en formato PDF, MP3 o MP4. También se definirá una fecha límite de entrega. Las tareas podrán calificarse individualmente, modificarse y/o eliminarse en cualquier momento, pero no se eliminarán automáticamente al llegar la fecha de entrega. En el apartado de Teoría, las tareas podrán sustituirse (o no) por un cuestionario tipo test. Dicho cuestionario permitirá la creación de preguntas con opciones de respuesta, y cada pregunta podrá ir acompañada de un archivo en formato MP3.

Asimismo, los alumnos podrán acceder a los mismos cuatro apartados (Ritmo, Entonación, Audición y Teoría). En cada apartado se mostrarán las mismas dos secciones:

- En la sección Libro del curso, los alumnos podrán visualizar el archivo PDF correspondiente y descargarlo, pero no modificarlo.
- En la sección Tareas a realizar, los alumnos podrán consultar la lista de tareas disponibles y realizar las entregas correspondientes. Los formatos permitidos serán PDF, MP3 o MP4, y/o bien un comentario textual dentro de la propia aplicación. En el caso de los cuestionarios, el alumno podrá

acceder de forma específica a cada uno para completarlos **apoyándose, si lo desea, en solo una pista por cada pregunta**. El cuestionario no se considerará entregado hasta que el alumno pulse expresamente el botón de envío.

Los alumnos y profesores dispondrán de un área específica de mensajes. En ella recibirán avisos cuando se cree, se modifique, se califique (alumno) o se entregue (profesor) una tarea o cuestionario, cuando reciban una calificación general (mensajes del sistema) o un mensaje del profesor.

3.3 OBJETIVOS FUNCIONALES DEL SISTEMA EXTENDIDO

Los objetivos funcionales descritos en el versión inicial se mantienen, ya que la esencia del sistema no cambia: facilitar la enseñanza del lenguaje musical con herramientas específicas de gestión y aprendizaje. No obstante, este complemento añade objetivos de evolución funcional centrados en la operatividad real y en la mejora de la experiencia del alumno:

- Permitir la administración masiva de usuarios y grupos mediante ficheros formateados, reduciendo la carga manual y los errores de alta.
- Completar la gestión del ciclo de vida de los usuarios (alumnos y profesores) agregando la posibilidad de revisarlos y eliminarlos y la de los grupos permitiendo eliminarlos.
- Incorporar mecanismos de apoyo al alumno durante la realización de cuestionarios para mejorar la experiencia de aprendizaje en actividades evaluables.

3.4 DIAGRAMA UML DE CASOS DE USO DEL SISTEMA EXTENDIDO

La ampliación funcional se refleja en el diagrama general de casos de uso del sistema. Para evitar confusión y mantener continuidad con el documento original, el diagrama se presentará con las nuevas incorporaciones destacadas visualmente mediante un código de color:

- **Rojo:** Caso de uso totalmente nuevo.
- **Azul:** Caso de uso existente sustituido (cambio de nombre y/o alcance).
- **Negro:** Caso de uso existente sin cambios de nombre (aunque pueda incorporar nuevos requisitos).

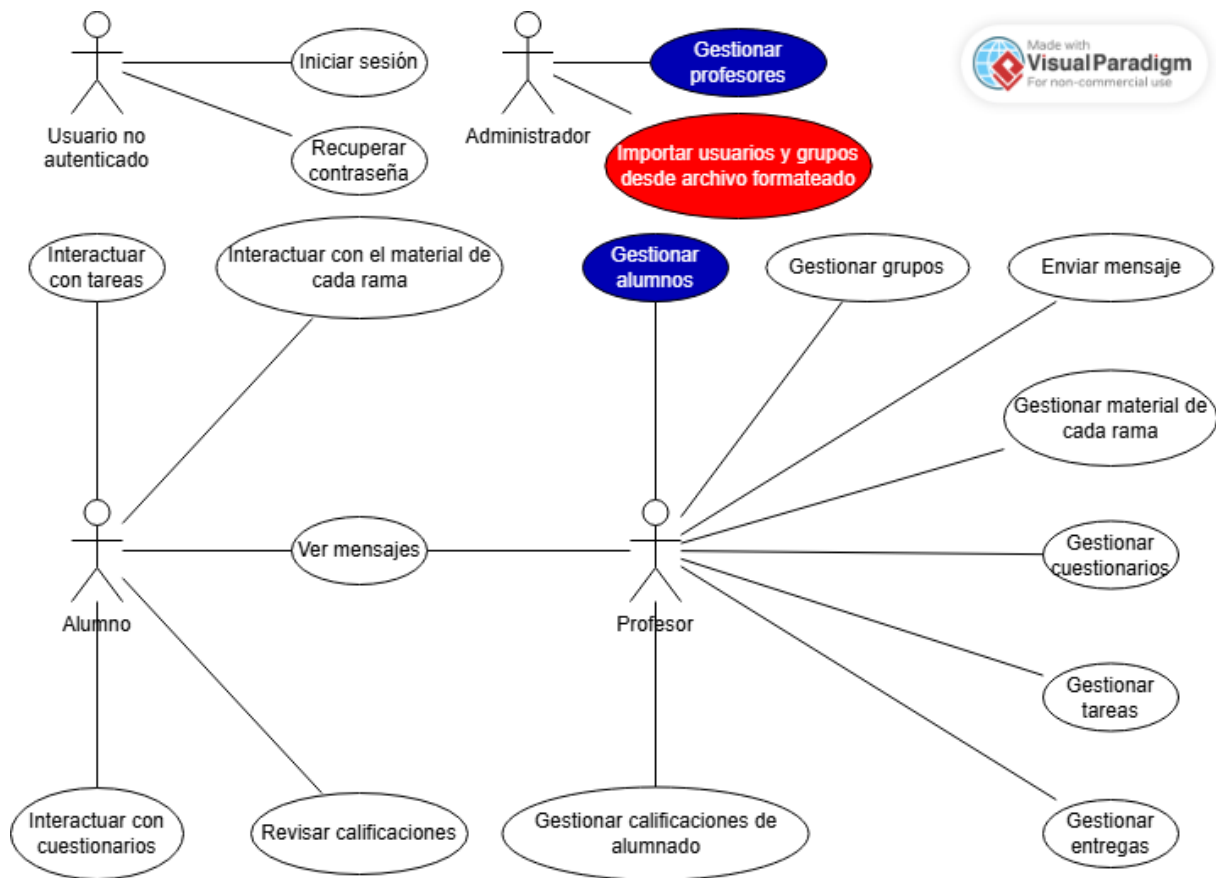


Figura 3.1: Diagrama UML de casos de uso del sistema extendido

3.5 DEFINICIÓN DEL DIAGRAMA DE CASOS DE USO

Como se detalló en el trabajo previo, el diagrama ofrece una visión objetiva pero todavía abstracta del sistema. Para alcanzar un nivel operativo y verificable, se definen a continuación únicamente los casos de uso nuevos y aquellos existentes que se ven modificados por la incorporación de requisitos adicionales, manteniendo el resto de definiciones de la versión inicial sin cambios. Cabe destacar que se ha mantenido el mismo código de color para el nombre del caso de uso y se ha usado el color rojo en el resto de campos de los casos de uso que no son totalmente nuevos para indicar en qué cambian.

3.5.1 Casos de uso nuevos o modificados

Caso de uso: Importar usuarios y grupos desde archivo formateado

Nombre	Importar usuarios y grupos desde archivo formateado
Descripción	Permitir al administrador crear usuarios y/o grupos mediante la carga de un fichero formateado, aplicando validaciones y creando relaciones de pertenencia según el contenido del archivo.
Actor	ACT-2
Requisitos funcionales	RF-31

Tabla 3.1: Caso de uso: Importar usuarios y grupos desde archivo formateado

Caso de uso que sustituye Crear profesor: Gestionar profesores

Nombre	Gestionar profesores
Descripción	Permitir al administrador crear, revisar y eliminar usuarios con el rol de profesor en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-2
Requisitos funcionales	RF-03 RF-04 RF-32 RF-33

Tabla 3.2: Caso de uso: Gestionar profesores

Caso de uso que sustituye Crear alumno: Gestionar alumnos

Nombre	Gestionar alumnos
Descripción	Permitir al profesor crear, revisar y eliminar usuarios con el rol de alumno en el sistema, proporcionando los datos necesarios y validando la información.
Actor	ACT-3
Requisitos funcionales	RF-05 RF-04 RF-34 RF-35

Tabla 3.3: Caso de uso: Gestionar alumnos

Caso de uso existente ampliado: Gestionar grupos

Nombre	Gestionar grupos
Descripción	Permitir al profesor crear nuevos grupos, revisar los que ha creado y elegir un grupo para trabajar con él o eliminarlo .
Actor	ACT-3
Requisitos funcionales	RF-06 RF-07 RF-08 RF-36

Tabla 3.4: Caso de uso: Gestionar grupos

Caso de uso existente ampliado: Interactuar con cuestionarios

Nombre	Interactuar con cuestionarios
Descripción	Permitir al alumno interactuar con los cuestionarios.
Actor	ACT-4
Requisitos funcionales	RF-18 RF-30 RF-37

Tabla 3.5: Caso de uso: Interactuar con cuestionarios**3.5.2 Requisitos funcionales nuevos**

En coherencia con el documento original, los identificadores continúan a partir del último requisito definido. A continuación se describen únicamente los nuevos requisitos y las ampliaciones necesarias de los casos de uso afectados.

RF-31	Importar usuarios y grupos desde archivo formateado
Descripción	Permitir al administrador crear usuarios y/o grupos mediante la carga de un fichero formateado, aplicando validaciones y creando relaciones de pertenencia según el contenido del archivo.
Precondición	El administrador debe estar autenticado y disponer de un fichero válido según el formato establecido.
Secuencia de pasos	1. El administrador accede a la opción de importación dentro del área de administración. 2. El administrador selecciona un fichero local para cargar. 3. El sistema valida el formato general del fichero (estructura, campos obligatorios y consistencia). 4. El sistema recorre los registros y valida datos (equivalente a RF-04 cuando aplique), incluyendo unicidad de correos y reglas de generación de credenciales. 5. El sistema crea los usuarios y grupos indicados si todos han sido validados. 6. El sistema informa del éxito.
Postcondición	Los usuarios y grupos válidos quedan registrados y listos para su uso.
Excepciones	■ El fichero no cumple el formato establecido o falta información obligatoria. ■ Existen registros con correos duplicados o datos inválidos. ■ Problemas de conexión con la base de datos.

Tabla 3.6: Requisito funcional: RF-31 Importar usuarios y grupos desde archivo formateado

RF-32	Revisar profesores
Descripción	Permitir al administrador visualizar la lista de profesores registrados en el sistema y acceder a su información básica para su gestión.
Precondición	El administrador debe estar autenticado y tener profesores creados.
Secuencia de pasos	1. El administrador entra en la sección de profesores. 2. El sistema recupera y muestra el listado de profesores registrados.
Postcondición	El administrador puede identificar profesores gestionables y realizar acciones de creación o eliminación desde el mismo contexto.
Excepciones	■ No existen profesores registrados. ■ Problemas de conexión a la base de datos.

Tabla 3.7: Requisito funcional: RF-32 Revisar profesores

RF-33	Eliminar profesor
Descripción	Permitir al administrador eliminar un profesor del sistema, eliminando todo su material asociado (tareas, cuestionarios, ramas, entregas y calificaciones), sus grupos y los alumnos creados bajo su responsabilidad, evitando referencias huérfanas o inconsistencias.
Precondición	El administrador debe estar autenticado y el profesor a eliminar debe existir.
Secuencia de pasos	1. El administrador accede a la administración de profesores y visualiza el listado. 2. El administrador selecciona un profesor y solicita su eliminación. 3. El sistema muestra confirmación. 4. El administrador confirma. 5. El sistema elimina al profesor y todos los datos relacionados con él. 6. El sistema informa del éxito.
Postcondición	El profesor deja de existir en el sistema y sus recursos asociados quedan gestionados de forma consistente.
Excepciones	■ El profesor no existe. ■ Problemas de conexión a la base de datos.

Tabla 3.8: Requisito funcional: RF-33 Eliminar profesor

RF-34	Revisar alumnos
Descripción	Permitir al profesor visualizar la lista de alumnos creados por él y acceder a su información básica para su gestión.
Precondición	El profesor debe estar autenticado y tener alumnos creados.
Secuencia de pasos	1. El profesor accede al área de administración y entra en la sección de alumnos. 2. El sistema recupera y muestra el listado de alumnos del profesor.
Postcondición	El profesor puede identificar a los alumnos gestionables y realizar acciones de creación o eliminación desde el mismo contexto.
Excepciones	■ No existen alumnos creados por el profesor. ■ Problemas de conexión a la base de datos.

Tabla 3.9: Requisito funcional: RF-34 Revisar alumnos

RF-35	Eliminar alumno
Descripción	Permitir al profesor eliminar un alumno creado por él, garantizando que el sistema gestiona de forma consistente la información asociada exclusivamente al alumno (calificaciones y entregas).
Precondición	El profesor debe estar autenticado y el alumno a eliminar debe haber sido creado previamente por el profesor.
Secuencia de pasos	1. El profesor accede al área de administración y visualiza el listado de alumnos gestionables. 2. El profesor selecciona un alumno y solicita su eliminación. 3. El sistema muestra un aviso de confirmación describiendo el alcance de la operación. 4. El profesor confirma la eliminación. 5. El sistema elimina el alumno, los datos exclusivos del alumno y las relaciones del alumno en el resto de registros que figure para mantener la consistencia. 6. El sistema informa del éxito.
Postcondición	El alumno deja de existir en el sistema y no es accesible desde la interfaz, a su vez, los datos asociados quedan en un estado consistente.
Excepciones	■ El alumno no existe o no ha sido creado por el profesor autenticado. ■ Problemas de conexión a la base de datos.

Tabla 3.10: Requisito funcional: RF-35 Eliminar alumno

RF-36	Eliminar grupo
Descripción	Permitir al profesor eliminar un grupo creado por él, gestionando de forma consistente todo el material asociado (tareas, cuestionarios, ramas, entregas y calificaciones).
Precondición	El profesor debe estar autenticado y el grupo a eliminar debe haber sido creado previamente por el profesor.
Secuencia de pasos	1. El profesor visualiza su listado de grupos y selecciona el grupo a eliminar. 2. El profesor solicita la eliminación del grupo. 3. El sistema solicita confirmación, indicando que se trata de una eliminación irreversible. 4. El profesor confirma. 5. El sistema elimina el grupo y todos los datos relacionados con el grupo menos los usuarios. 6. El sistema informa del éxito.
Postcondición	El grupo deja de existir en el sistema y no puede seleccionarse, además, no quedan recursos asociados accesibles de forma inconsistente.
Excepciones	■ El grupo no existe o no ha sido creado por el profesor autenticado. ■ Problemas de conexión a la base de datos.

Tabla 3.11: Requisito funcional: RF-36 Eliminar grupo

RF-37	Solicitar pista durante un cuestionario
Descripción	Permitir al alumno solicitar solo una pista asociada a una pregunta concreta durante la realización de un cuestionario, recibiendo una orientación limitada que facilite el razonamiento sin revelar la respuesta.
Precondición	El alumno debe estar autenticado, haber accedido a un cuestionario activo y encontrarse en el proceso de respuesta.
Secuencia de pasos	1. El sistema muestra las preguntas del cuestionario. 2. El alumno solicita una pista para la pregunta que tenga dudas. 3. El sistema presenta una pista contextual (orientación), sin mostrar directamente la opción correcta. 4. El alumno continúa respondiendo a la pregunta. 5. El sistema permite finalizar y entregar el cuestionario como en RF-30.
Postcondición	El alumno recibe una pista y puede continuar la realización de la entrega bajo las reglas de RF-30.
Excepciones	■ El cuestionario no está activo o el alumno no tiene acceso al mismo. ■ El servicio de generación de pista no está disponible. ■ Problemas de conexión a la base de datos.

Tabla 3.12: Requisito funcional: RF-37 Solicitar pista durante un cuestionario

3.6 REQUISITOS NO FUNCIONALES NUEVOS

El proyecto original definía un requisito de portabilidad (RNF-04) orientado a asegurar que la aplicación pudiera utilizarse en distintos dispositivos y entornos. En este complemento, esa portabilidad se concreta en un objetivo más exigente y verificable: la exportación efectiva a un entorno móvil Android, aprovechando el diseño híbrido y responsive ya adoptado.

RNF-08	Portabilidad a Android
Descripción	El sistema deberá poder ejecutarse como aplicación instalada en dispositivos Android, manteniendo las funcionalidades esenciales y una experiencia de usuario consistente.
Justificación	Se da continuidad a RNF-04 (Portabilidad) del proyecto base, materializando la ejecución en un entorno móvil real para reforzar accesibilidad y facilitar su implantación en contextos educativos.
Criterio de cumplimiento	La aplicación se exporta y ejecuta en Android mediante herramientas de construcción móvil, pudiendo autenticarse, navegar por ramas, consultar materiales y realizar acciones principales sin degradación funcional.

Tabla 3.13: Requisito no funcional: RNF-08 Portabilidad a Android

3.7 TECNOLOGÍAS Y ARQUITECTURA DE LA SOLUCIÓN EXTENDIDA

La solución extendida mantiene la arquitectura global cliente-servidor basada en una API REST y una organización inspirada en el patrón MVC (Modelo-Vista-Controlador) idéntica en su planteamiento a la versión inicial. Esta continuidad tecnológica no es accidental: la pila original proporciona un equilibrio adecuado entre modularidad, escalabilidad y facilidad de mantenimiento, lo que permite incorporar nuevas capacidades sin necesidad de reestructurar el sistema ni introducir deuda técnica.

3.7.1 Pila tecnológica heredada del proyecto base

El proyecto se apoya en un conjunto de tecnologías consolidadas que cubren de manera eficaz las necesidades de una aplicación educativa con múltiples roles, gestión de contenidos multimedia y comunicación bidireccional:

- **Ionic + Angular (Frontend).** Se mantiene un frontend híbrido desarrollado con Ionic y Angular, que permite construir una interfaz responsive y reutilizable tanto en entorno web como móvil. Angular aporta una estructura basada en componentes, servicios y módulos que facilita la organización del código, la reutilización de lógica y la separación entre presentación y lógica de negocio, mientras que Ionic proporciona componentes visuales optimizados para dispositivos táctiles y móviles.
- **Node.js + Express (Backend).** El backend continúa implementándose mediante un servidor Node.js con Express, exponiendo una API REST encargada de centralizar la lógica de negocio, la validación de datos y la aplicación de reglas de acceso. Esta combinación ofrece un entorno ligero y flexible, adecuado para aplicaciones con múltiples operaciones CRUD y consumo intensivo de recursos multimedia.
- **MongoDB (Persistencia).** Se mantiene una base de datos documental MongoDB para almacenar los datos que requieren de persistencia. El modelo no relacional facilita la evolución del esquema de datos y permite representar relaciones entre entidades educativas sin penalizar la flexibilidad, algo especialmente relevante en un sistema que crece de forma incremental. También se hace uso de MongoDB Atlas, un servicio gestionado en la nube que ofrece una solución escalable y segura.
- **JWT (Autenticación y autorización).** La autenticación se sigue gestionando mediante JSON Web Tokens, permitiendo un control claro de sesiones y permisos en función del rol del usuario. Este enfoque se adapta bien a una arquitectura REST y evita mantener estado de sesión en el servidor.
- **Swagger / OpenAPI (Documentación).** La documentación de la API se mantiene mediante Swagger, proporcionando una especificación clara y verificable de los endpoints disponibles, útil tanto para pruebas como para mantenimiento futuro.
- **Testing (Jest/Supertest y Jasmine/Karma).** Se conserva el uso de Jest y Supertest para pruebas del backend, y Jasmine/Karma para el frontend, permitiendo validar de manera automatizada el comportamiento del sistema y reducir regresiones durante la evolución del proyecto.



- **Integración y despliegue continuo (Render, GitHub Actions y Pages).** Se continúa ejecutando el flujo de integración y despliegue continuo en GitHub Actions para automatizar el testing y el despliegue, que será realizado en Render para el backend y GitHub Pages para el frontend.

3.7.2 Tecnologías incorporadas en la solución extendida

La extensión del sistema introduce nuevas tecnologías que amplían las capacidades técnicas de la aplicación sin modificar su arquitectura base ni los principios de diseño definidos en el trabajo previo. Estas tecnologías se integran como componentes complementarios, manteniendo el desacoplamiento y la coherencia estructural del sistema.

Servicio de inferencia externa mediante Groq

Se incorpora un servicio de inferencia externa utilizando Groq [1] como proveedor, haciendo uso de su API pública para el envío de peticiones desde el backend del sistema. En este contexto, Groq actúa como un servicio externo capaz de procesar *prompts* de inteligencia artificial y devolver respuestas generadas a partir de modelos de lenguaje, sin que dichos modelos formen parte de la infraestructura propia de la aplicación.

Desde un punto de vista tecnológico, la utilización de la API de Groq implica la integración de un servicio HTTP externo que recibe solicitudes estructuradas, incluyendo el contenido del *prompt*, y devuelve una respuesta generada en un formato estándar (por ejemplo, JSON). Esta aproximación permite tratar el sistema de inferencia como un recurso remoto, consumido de forma análoga a cualquier otro servicio web.

El uso de una API de inferencia externa presenta varias implicaciones técnicas relevantes:

- Permite externalizar el procesamiento asociado a modelos de lenguaje, evitando la necesidad de desplegar, mantener o escalar infraestructura específica para inferencia.
- Facilita el control del flujo de peticiones, ya que las llamadas a la API pueden regularse, validarse y encapsularse desde el backend.
- Aísla la lógica de generación de contenido del resto del sistema, permitiendo modificar el proveedor o el modelo utilizado sin afectar a la arquitectura interna.

Desde el punto de vista del diseño software, la integración mediante una API externa refuerza el desacoplamiento del sistema, ya que el servicio de inferencia se consume como una dependencia claramente delimitada, manteniendo la coherencia con la arquitectura por capas adoptada en el proyecto.



Figura 3.2: Logo Groq

Capacitor como evolución del soporte móvil híbrido

El uso de Ionic junto con Angular permite abordar el desarrollo multiplataforma mediante una única base de código. Este enfoque resuelve gran parte de los problemas asociados a la portabilidad de la interfaz y de la lógica de la aplicación, tanto en entorno web como en dispositivos móviles. No obstante, el desarrollo puramente web presenta una limitación clara: existen funcionalidades propias del sistema operativo móvil que no pueden ser gestionadas directamente desde el navegador.

Primeros intentos de integración nativa. Para cubrir esta necesidad surgió Cordova [2], cuya función principal era actuar como un puente entre la aplicación web y el dispositivo. Gracias a esta tecnología fue posible acceder desde JavaScript a funcionalidades nativas, permitiendo que las aplicaciones híbridas pudieran comportarse como aplicaciones móviles instaladas. En su contexto histórico, Cordova supuso un avance clave para validar el modelo híbrido.

Sin embargo, Cordova respondía a una etapa temprana de esta evolución. A medida que las aplicaciones comenzaron a requerir mayor control y personalización del entorno móvil, sus limitaciones se hicieron evidentes.

Evolución hacia un modelo más flexible. Capacitor [3] aparece como la evolución natural de este enfoque, manteniendo la reutilización del código web, pero adaptándolo a las necesidades actuales del desarrollo móvil. La diferencia entre ambos modelos puede resumirse claramente en los siguientes puntos:

- **Relación con el código nativo:** Cordova ocultaba en gran medida la capa nativa, mientras que Capacitor la integra como parte explícita del proyecto.
- **Configuración del sistema:** en Cordova, los ajustes específicos resultaban complejos y poco transparentes; Capacitor facilita configuraciones controladas y mantenibles.
- **Evolución y mantenimiento:** Capacitor se alinea mejor con las herramientas y prácticas actuales del desarrollo móvil, favoreciendo la sostenibilidad del proyecto.

Papel actual de Capacitor en el proyecto. En la actualidad, Capacitor no se limita únicamente al empaquetado de la aplicación y a su ejecución como aplicación instalada, funciones que ya desempeñaba Cordova. Su principal aportación es actuar como una capa de integración que permite extender el frontend híbrido con capacidades propias del entorno móvil cuando el proyecto lo requiere.

En el contexto de este proyecto, Capacitor resulta especialmente relevante porque:

- Permite generar una aplicación móvil instalable a partir del frontend híbrido existente.
- Mantiene una única base de código, garantizando coherencia funcional entre la versión web y la versión móvil.
- Facilita la incorporación de funcionalidades y configuraciones específicas del dispositivo sin alterar la arquitectura del sistema.

De este modo, Capacitor se incorpora como un elemento habilitador que amplía el alcance de la solución, facilitando su implantación real en dispositivos móviles y manteniendo los principios de reutilización, modularidad y separación de responsabilidades definidos en el proyecto.



Figura 3.3: Logo Capacitor

Android Studio y entorno de construcción móvil

La tercera tecnología incorporada es Android Studio [4], utilizado como entorno de construcción, empaquetado y validación de la aplicación en dispositivos Android. Su inclusión se apoya en el carácter híbrido del frontend, que permite reutilizar la misma base de código web para generar una aplicación instalable.

Desde el punto de vista tecnológico, el uso de Android Studio implica:

- La integración con la cadena de herramientas oficial de Android para la compilación, firma y despliegue de aplicaciones.
- La generación de un contenedor nativo que encapsula la aplicación híbrida, permitiendo su ejecución como aplicación instalada.
- El acceso a herramientas de depuración, emulación y análisis propias del ecosistema Android.

Este enfoque no introduce una nueva base de código ni una bifurcación tecnológica, sino que extiende el alcance del sistema aprovechando el diseño *responsive* existente. De este modo, la portabilidad se aborda como una extensión técnica natural del frontend híbrido, sin duplicar lógica ni comprometer la consistencia del sistema.



Figura 3.4: Logo Android Studio.

En conjunto, las tecnologías incorporadas refuerzan la arquitectura existente, ampliando las capacidades del sistema desde un punto de vista estrictamente técnico y manteniendo los principios de modularidad, reutilización y separación de responsabilidades definidos en el trabajo previo.



A nivel conceptual, la arquitectura global permanece idéntica. Se incluye a continuación el diagrama original para facilitar la lectura del documento sin obligar a consultar constantemente la versión original.

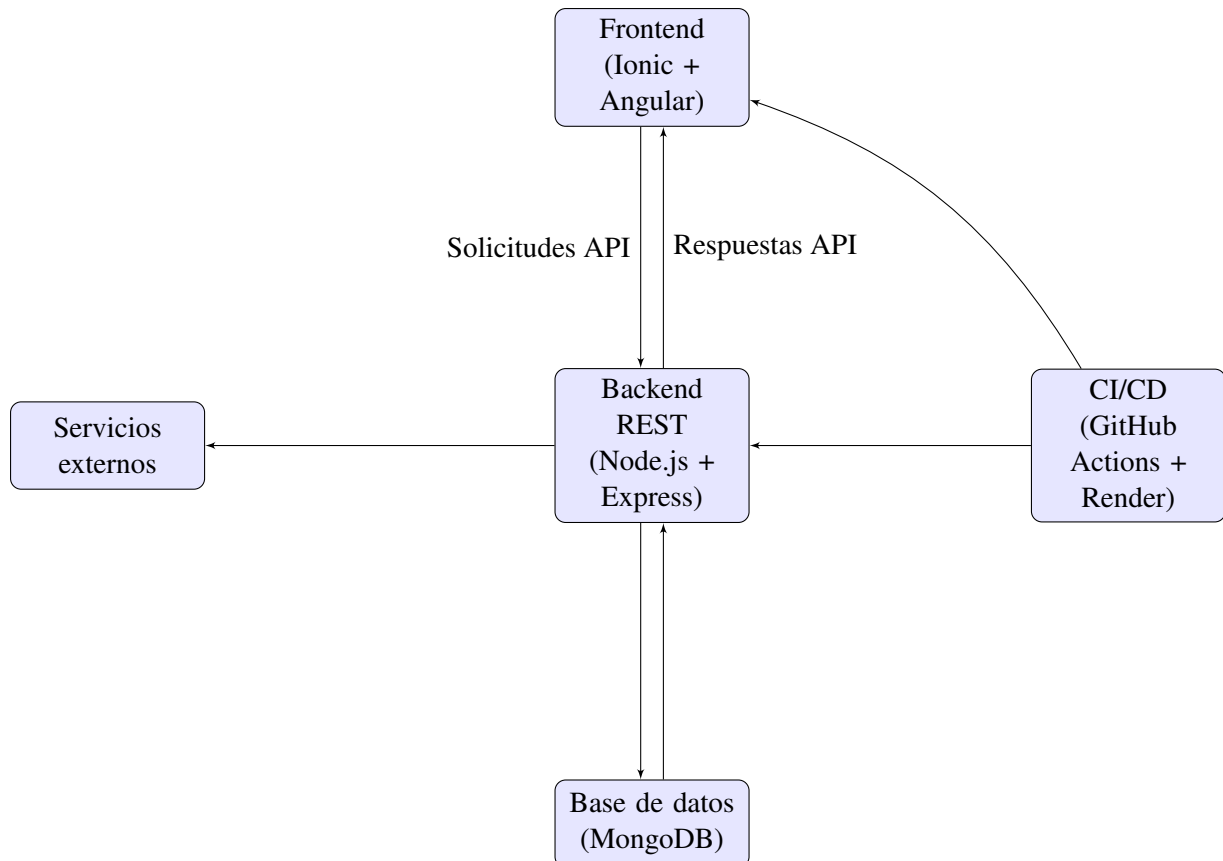


Figura 3.5: Arquitectura global de la aplicación (sin cambios respecto al proyecto base)

Con esta definición, la solución extendida queda descrita con el mismo enfoque de la primera versión: continuidad en arquitectura y metodología, y ampliación funcional bien acotada, verificable y orientada a la implantación real.

4 PLANIFICACIÓN DEL PROYECTO

La planificación de este complemento se concibe como una extensión directa de la planificación definida en el proyecto original, manteniendo sus mismos criterios generales de organización, seguimiento y gestión del tiempo. Dado que la base del sistema ya se encuentra diseñada e implementada, no resulta necesario replantear la estructura global del cronograma, sino ampliarla de manera coherente para incorporar las nuevas tareas derivadas de la evolución funcional del proyecto.

En este contexto, la planificación del complemento se centra en integrar las actividades adicionales asociadas a la mejora de funcionalidades, la incorporación de nuevas capacidades y la preparación del sistema para su implantación, respetando la secuencia lógica y la continuidad del trabajo previamente realizado. De este modo, se garantiza que la evolución del proyecto se produce ordenada y alineada con los objetivos ya establecidos.

El diagrama Gantt de planificación reutiliza la estructura del cronograma original, incorporando las nuevas tareas y sub-tareas de manera diferenciada mediante un código de color, lo que permite identificar claramente qué actividades pertenecen al proyecto inicial y cuáles corresponden específicamente a este complemento. Esta representación facilita la comprensión del alcance real de la extensión sin introducir redundancias en la documentación.

En conjunto, la planificación del complemento actúa como un marco de referencia que guía la ejecución de esta fase de evolución del proyecto, manteniendo la flexibilidad propia del contexto académico y asegurando la coherencia con el trabajo previamente desarrollado.

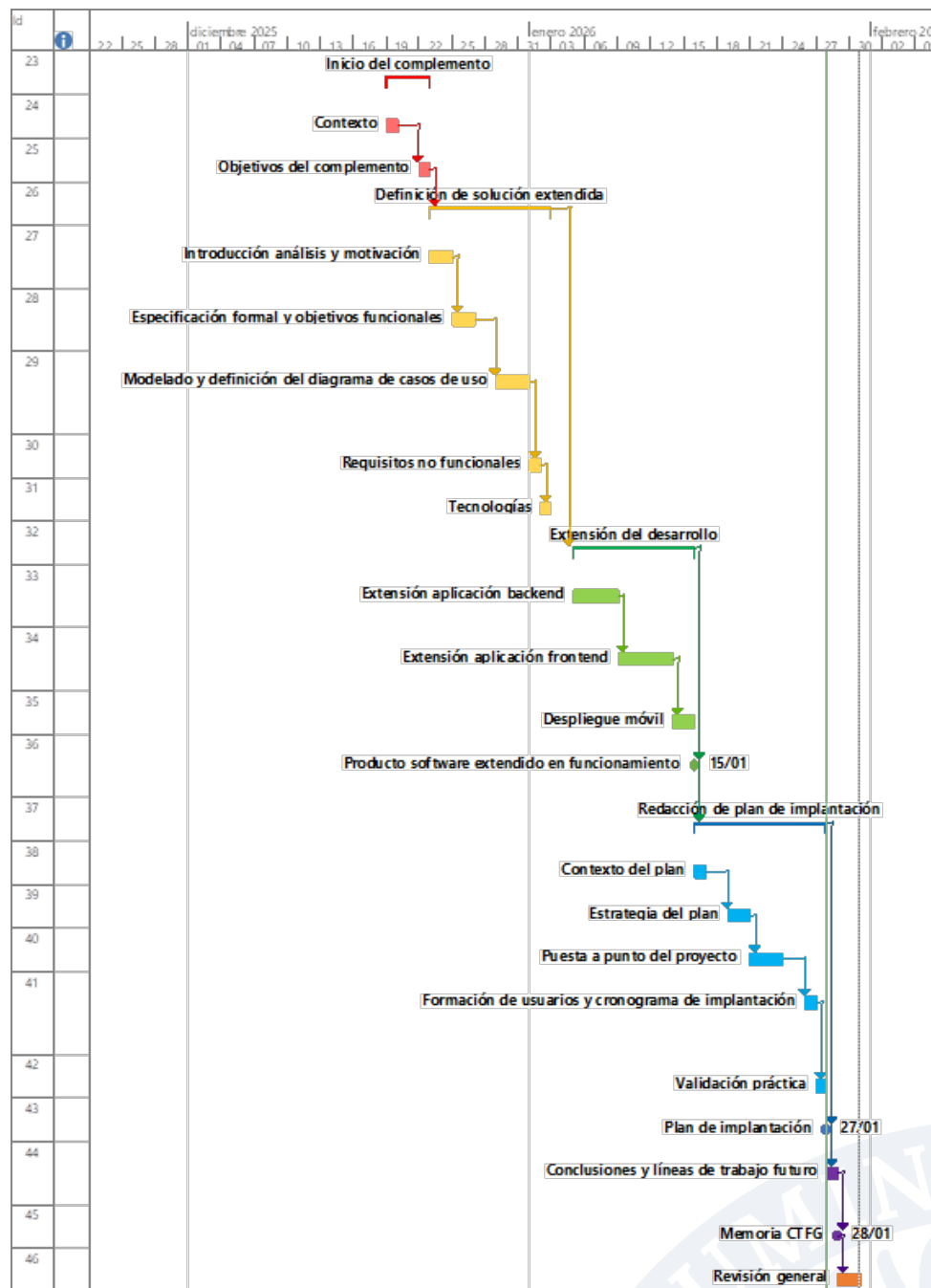


Figura 4.1: Planificación del proyecto con nuevas tareas destacadas

Por último, también se añade la planificación de las tareas principales en horas.

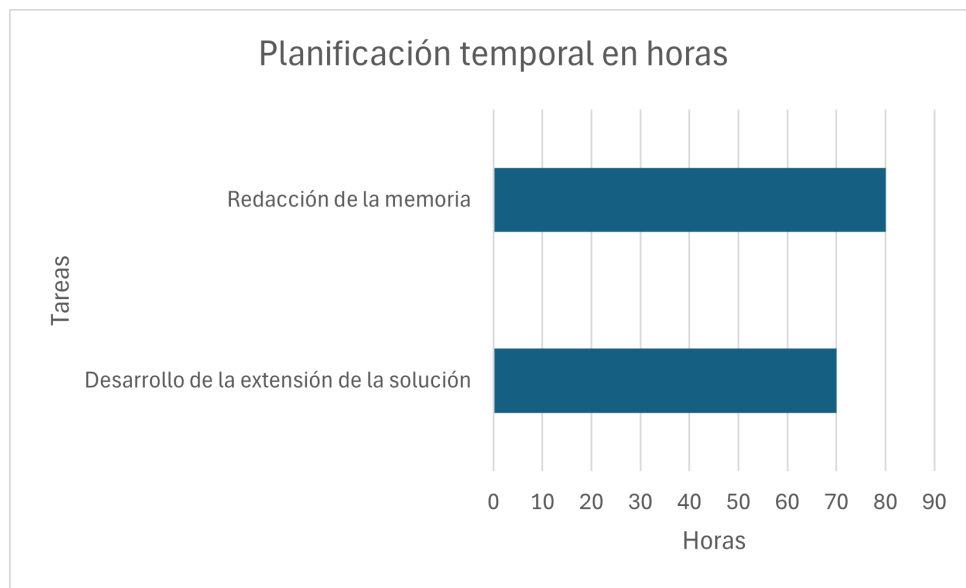


Figura 4.2: Planificación de las tareas principales en horas

5 DESARROLLO DE LA APLICACIÓN BACKEND

El backend de este complemento se apoya directamente en la base implementada en versión inicial, manteniendo la misma pila tecnológica: **Node.js** como entorno de ejecución y **Express** como framework para exponer una **API REST**. A nivel de organización interna, se conserva la separación de responsabilidades típica de la versión anterior, con una estructura inspirada en **MVC adaptado a backend** (sin capa de interfaz): **modelos** para persistencia y esquema de datos, **controladores** para la lógica de operaciones y **rutas** como capa de entrada a los endpoints.

En esta fase del complemento, la mayor diferencia no está en “hacer un CRUD nuevo”, sino en reforzar operaciones de uso real (revisiones, eliminaciones y altas masivas) que requieren más validación, limpieza de relaciones y control de errores. Además, aparece una pieza nueva dentro del backend: **servicios**, usados para encapsular integraciones externas como la comunicación con la **API de Groq**.

5.1 MODELADO DE DATOS

Con el objetivo de proporcionar una visión global de la estructura de la información manejada por el sistema, se elaboró en el sistema base un modelo conceptual de datos que identifica las principales entidades del dominio y las relaciones existentes entre ellas. Este modelo permite comprender de forma clara cómo se organiza la información dentro de la aplicación y sirve como referencia para la posterior implementación de la persistencia de datos en el backend.

Es importante recordar que el diagrama presentado no corresponde a un diagrama Entidad-Relación (ER) en sentido estricto, en su lugar, se adopta un enfoque conceptual que refleja la estructura lógica de los datos y sus relaciones semánticas, independientemente de su implementación física.

La Figura 5.1 muestra el modelo conceptual de datos del sistema, proporcionando una visión estructural que complementa la definición funcional y arquitectónica presentada en los apartados anteriores.

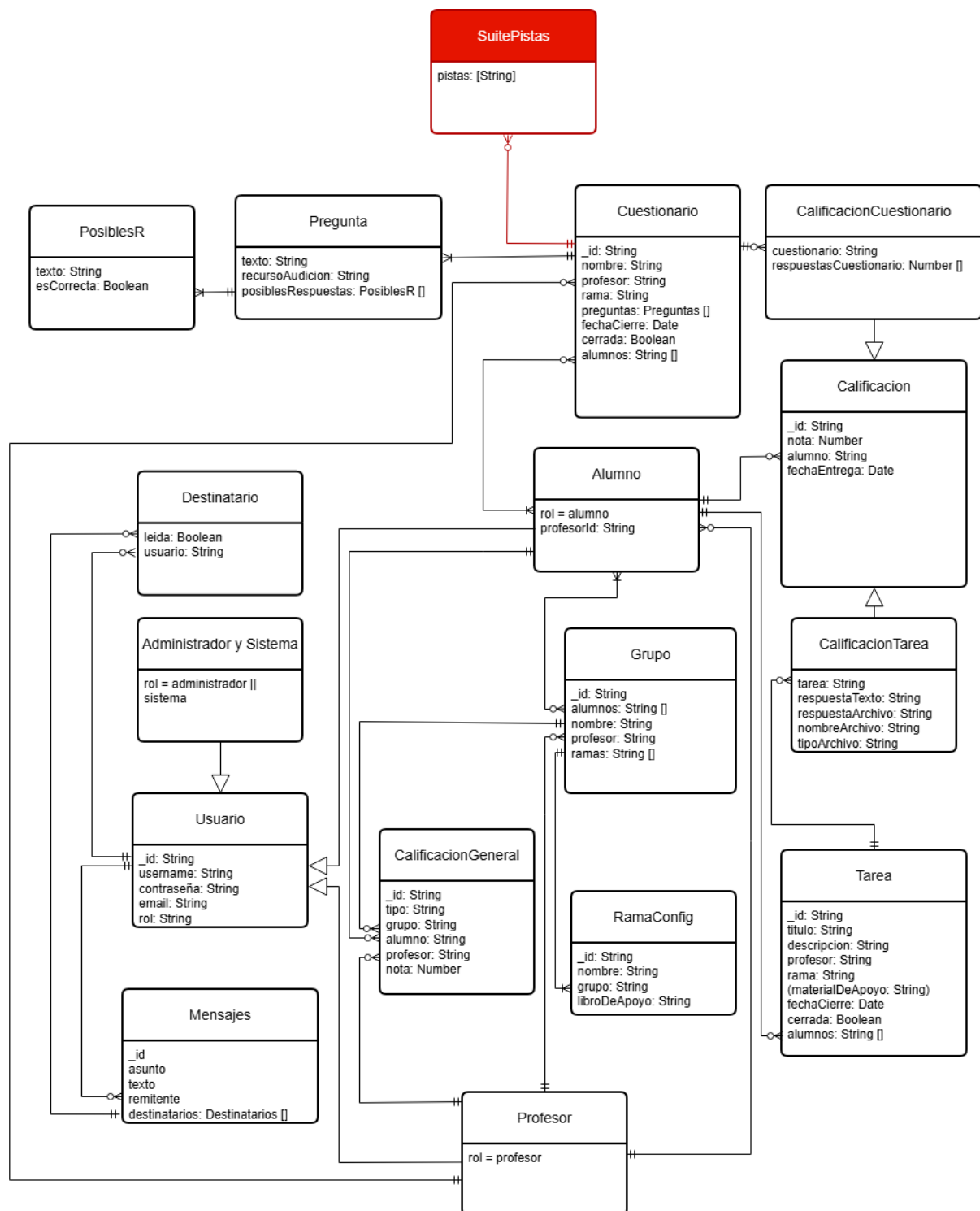


Figura 5.1: Modelo conceptual de datos del sistema

5.2 IMPLEMENTACIÓN

5.2.1 Operaciones lógicas (revisión y eliminación)

Las operaciones de revisión (listados y consultas) reutilizan el mismo patrón del trabajo previo: filtrado por rol, respuesta en JSON y acceso a través de controladores. En cambio, las eliminaciones se han ampliado para contemplar el efecto real en otras colecciones (por ejemplo: al eliminar un grupo se eliminan/limpian tareas, cuestionarios, entregas y calificaciones asociadas, y se elimina la relación en los alumnos).

Se incluye un ejemplo representativo (el resto de eliminaciones sigue una estructura muy similar):

```
// Ejemplo representativo: eliminar un grupo y limpiar datos asociados
async function eliminarGrupo(groupId, profesorId) {
  //1) Verificar que el grupo existe y pertenece al profesor autenticado
  const grupo = await Grupo.findOne({
    _id: groupId,
    profesor: profesorId
  });

  if (!grupo) {
    throw new Error('Grupo no encontrado o sin permisos');
  }

  //2) Eliminar contenido asociado al grupo
  await Tarea.deleteMany({ grupo: groupId });
  await Cuestionario.deleteMany({ grupo: groupId });

  //3) Eliminar entregas y calificaciones asociadas
  await Entrega.deleteMany({ grupo: groupId });
  await Calificacion.deleteMany({ grupo: groupId });

  //4) Eliminar el documento principal
  await Grupo.deleteOne({ _id: groupId });
  // Las relaciones también quedan eliminadas
  return { ok: true };
}
```

5.2.2 Importación masiva desde CSV (normalización, validación y creación)

La importación masiva desde CSV introduce un flujo distinto al alta individual, ya que los datos provienen de una fuente externa y pueden contener errores, duplicados o referencias inconsistentes. Por este motivo, el proceso se ha diseñado siguiendo un enfoque estrictamente controlado, dividido en dos fases claramente diferenciadas.

- **Validación completa previa:** El archivo CSV se parsea y se recorren todas las filas sin crear ningún elemento en la base de datos. Durante esta fase se validan campos obligatorios, duplicados

internos, referencias cruzadas entre usuarios y grupos, y se comprueba la coherencia con las reglas de negocio del sistema.

- **Creación atómica:** Únicamente si la validación finaliza sin errores, se procede a la creación de usuarios y grupos dentro de una transacción de Mongoose, garantizando que el proceso se complete íntegramente o se revierta completamente en caso de fallo.

Además, se aplica una regla del sistema clave en este complemento: **un alumno pertenece a un único profesor**. Esto se comprueba antes de crear los grupos: si un grupo de un profesor intenta incluir un alumno asignado a otro profesor, se detiene la importación.

Este enfoque permite detectar todos los errores del CSV de forma anticipada y evita la creación parcial de datos, asegurando la consistencia del sistema incluso ante archivos mal formateados o incompletos. Dado que la implementación completa de este proceso resulta extensa y muy dependiente de detalles técnicos, se ha optado por presentar un código conceptual que resume el flujo principal, facilitando su comprensión sin perder el significado funcional.

```
func importarDesdeCSV(archivoCSV):  
  
    filas <- parsearCSV(archivoCSV)  
    errores <- []  
  
    // Fase 1: validación completa  
    para cada fila en filas:  
        validarCamposObligatorios(fila)  
        validarDuplicadosInternos(fila)  
        validarReferencias(fila)  
        aplicarReglasDeNegocio(fila)  
        acumularErrores(errores)  
  
    si errores no está vacío:  
        devolver errores y cancelar proceso  
  
    // Fase 2: creación atómica  
    iniciarTransacción()  
  
    crearProfesores()  
    crearAlumnos(asignados a su profesor)  
    crearGrupos(verificando coherencia)  
  
    confirmarTransacción()  
  
    enviarCorreosConCredenciales()  
    devolver resultado de éxito
```

5.2.3 Servicio de IA con Groq (integración por API y capa de servicios)

En el proyecto base, la lógica se repartía principalmente entre **rutas**, **controladores** y **modelos**. En este complemento se introduce además una **capa de servicios** para encapsular integraciones externas. Groq se integra mediante su **API pública**, y en la práctica el backend envía **prompts** (texto estructurado con instrucciones y contexto) y recibe una respuesta generada. Separarlo en un servicio evita mezclar esta comunicación externa con la lógica interna del sistema.

```
// Importación del SDK oficial de Groq para realizar peticiones
// al servicio de inferencia mediante su API
const Groq = require('groq-sdk');

// Inicialización del cliente de Groq utilizando la clave
// de API definida en las variables de entorno
const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });

// Función asíncrona encargada de generar una pista teórica
// breve para una pregunta tipo test
async function generarPistaTeoria({
  preguntaTexto,
  posiblesRespuestas,
  recursoAudicion
}) {

  // Comprobación previa de la existencia de la clave de API.
  // Si no está configurada, se interrumpe la ejecución
  if (!process.env.GROQ_API_KEY) {
    throw new Error('GROQ_API_KEY no está configurada');
  }

  // Construcción del bloque de opciones en formato texto.
  // Se numeran las respuestas y se evita provocar errores
  // en caso de que algún elemento no tenga texto definido
  const opciones = Array.isArray(posiblesRespuestas)
    ? posiblesRespuestas
      .map((r, i) => `${i + 1}) ${r?.texto ?? ''}`)
      .join('\n')
    : '';

  // Definición del contenido que se enviará al modelo.
  // Incluye la pregunta, las opciones y una serie de
  // restricciones para controlar el tipo de respuesta
  const userContent =
    `Genera UNA pista breve para ayudar a responder esta pregunta tipo test`
```

sin revelar la respuesta.

Pregunta:

`${preguntaTexto}`

Opciones:

`${opciones}`

`${recursoAudicion ? 'Hay un recurso de audición asociado.' : ''}`

Requisitos:

- 1 o 2 frases máximo
- No digas cuál es la opción correcta
- No des la solución ni pasos completos
- No menciones "IA" ni "modelo";

```
// Envío de la petición al servicio de Groq indicando
// el modelo, los mensajes de contexto y los parámetros
// que controlan la generación de la respuesta
```

```
const completion = await groq.chat.completions.create({
  model: 'llama-3.3-70b-versatile',
  messages: [
    { role: 'system', content: 'Eres un tutor de música.
    Das pistas breves y seguras.' },
    { role: 'user', content: userContent }
  ],
  temperature: 0.2,
  max_tokens: 90
});
```

```
// Extracción segura del contenido generado por el modelo.
// Se utiliza encadenamiento opcional para evitar errores
// si la respuesta no tiene la estructura esperada
const hint =
```

```
  completion?.choices?.[0]?.message?.content?.trim();
```

```
// Validación final para asegurar que se ha generado una pista
if (!hint) throw new Error('No se pudo generar pista');
```

```
// Limpieza del texto devuelto antes de retornarlo,
// eliminando comillas o espacios sobrantes
return hint
```

```
  .replace(/^[\"'\s]+|[/\"'\s]+$/g, '')
  .trim();
```

```
}
```

```
// Exportación de la función para su reutilización
// desde otras partes del backend
module.exports = { generarPistaTeoria };
```

5.3 TESTING

El backend mantiene el enfoque de testing definido en el proyecto base, utilizando **Jest** como framework de pruebas unitarias y **Supertest** como herramienta para la validación de endpoints HTTP. Tal como se describió previamente en el proyecto base, esta combinación permite comprobar sistemáticamente la correcta implementación de la lógica de negocio, la validez de las respuestas REST y el cumplimiento de las reglas funcionales del sistema.

Sobre esta base consolidada, la solución extendida refuerza el conjunto de pruebas automatizadas con nuevos escenarios derivados de las funcionalidades incorporadas, prestando especial atención a aquellos flujos que presentan una mayor complejidad lógica o impacto sobre la integridad de los datos.

En particular, se ha puesto el foco en validar:

- Que los procesos de importación masiva desde CSV detecten correctamente los errores de formato, referencias inconsistentes y violaciones de reglas de negocio, sin generar datos parciales;
- Que las importaciones válidas produzcan la creación íntegra y coherente de usuarios y grupos;
- Que las operaciones sensibles, como la eliminación de usuarios o la generación de pistas mediante servicios externos, respondan adecuadamente según el contexto, los permisos y el estado del sistema.

Este enfoque garantiza que la ampliación funcional no comprometa la estabilidad global del backend y que los nuevos flujos queden correctamente integrados dentro de la arquitectura existente.

Para ello, se mantiene la estrategia de aislamiento descrita en la versión inicial del proyecto, mediante el uso intensivo de *mocks* y *spies*, lo que permite simular de forma controlada la interacción con servicios externos, controladores internos y acceso a base de datos. Asimismo, el uso de Supertest facilita la ejecución de peticiones HTTP reales contra la aplicación, permitiendo validar tanto los códigos de estado como el contenido completo de las respuestas.

Como fragmento representativo, se presenta a continuación el conjunto de pruebas desarrollado para el endpoint encargado de devolver una pista asociada a una pregunta concreta, accesible mediante la ruta GET /cuestionarios/:id/preguntas/:index/pista. Este endpoint introduce un nuevo flujo funcional basado en la integración con un servicio externo de inferencia, por lo que resulta especialmente relevante validar tanto los escenarios normales como las situaciones de error controlado.

```
// Bloque de pruebas para el endpoint encargado de
// devolver una pista asociada a una pregunta concreta
```

```
describe('GET /cuestionarios/:id/preguntas/:index/pista', () => {

  // Caso de éxito: la petición es válida y se devuelve
  // una pista correctamente generada
  it('200 - devuelve pista', async () => {

    // Simulación del comportamiento del controlador para
    // evitar dependencias externas durante el test
    jest.spyOn(cuestionarioController, 'getPistaPregunta')
      .mockResolvedValue({
        status: 200,
        body: { pista: 'PISTA', cached: true }
      });

    // Ejecución de la petición HTTP contra la aplicación
    // utilizando Supertest
    const res = await request(app)
      .get('/cuestionarios/c1/preguntas/0/pista');

    // Verificación del código de estado y del contenido
    // de la respuesta
    expect(res.status).toBe(200);
    expect(res.body.pista).toBe('PISTA');
  });

  // Caso de error: el índice de la pregunta no es válido
  // y no puede ser interpretado como un número
  it('400 - preguntaIndex inválido', async () => {

    // Petición con un índice incorrecto para comprobar
    // la validación de parámetros en la ruta
    const res = await request(app)
      .get('/cuestionarios/c1/preguntas/abc/pista');

    // Se espera una respuesta de error por parte del servidor
    expect(res.status).toBe(400);
  });

  // Caso de error controlado: el servicio de generación
  // de pistas no está disponible temporalmente
  it('503 - mantenimiento', async () => {

    // Simulación de una respuesta del controlador que
    // indica indisponibilidad del servicio externo
    jest.spyOn(cuestionarioController, 'getPistaPregunta')
      .mockResolvedValue({
```

```
        status: 503,  
        body: { error: 'Pistas en mantenimiento.' }  
    });  
  
    // Ejecución de la petición en un escenario de  
    // mantenimiento del servicio  
    const res = await request(app)  
        .get('/cuestionarios/c1/preguntas/0/pista');  
  
    // Comprobación de que el código de estado refleje  
    // correctamente la situación de error  
    expect(res.status).toBe(503);  
    });  
});
```

Este conjunto de pruebas permite validar de manera precisa el correcto funcionamiento del endpoint bajo distintos escenarios, asegurando una integración fiable con el servicio externo de inferencia y una gestión coherente de los errores. De este modo, se refuerza la robustez del backend y se garantiza que la nueva funcionalidad se incorpora sin comprometer la estabilidad global del sistema.

6 DESARROLLO DE LA APLICACIÓN FRONTEND

El frontend del complemento se mantiene sobre la pila original: **Angular + Ionic**. Angular aporta la estructura modular basada en **componentes**, **servicios** y **módulos**, mientras que Ionic proporciona componentes visuales pensados para interfaz táctil y ejecución híbrida.

A nivel de patrón, el desarrollo sigue el esquema habitual de Angular (frecuentemente descrito como un **MVC/MVVM práctico**): los **componentes** gestionan estado y renderizado de la vista, los **servicios** encapsulan lógica reutilizable y comunicación HTTP, y la “capa modelo” se representa mediante interfaces/DTOs y estructuras que viajan entre frontend y backend.

En este complemento, aunque se han añadido pantallas y funcionalidades, el cambio más visible y relevante en frontend es la **exportación real a Android** y los ajustes necesarios para que la aplicación se comporte bien como app instalada.

6.1 IMPLEMENTACIÓN

6.1.1 Exportación a Android con Capacitor

La exportación a móvil se realiza con **Capacitor**, que actúa como puente entre el proyecto web y el proyecto nativo Android. A nivel práctico:

- Genera un contenedor Android para ejecutar la app como instalada,
- Permite usar plugins para acceder a capacidades del dispositivo,
- Facilita ajustes que en navegador (PC) no son necesarios.

Ajuste de menús del sistema

En sistemas móviles pueden aparecer solapes con algunos menús nativos como la barra de estado superior. Actualmente, Capacitor en combinación con Ionic, ofrecen una solución para ello bastante sencilla gracias a su conjunto de plugins, además también se aplican soluciones de padding para no solapar con otros menús como la botonera inferior del sistema nativo.

```
// Platform, servicio Standalone de Ionic
// que detecta si el entorno en ejecución es web o móvil (Android o iOS)
if (this.platform.is('capacitor')) {
  // StatusBar, plugin de Capacitor que permite modificar
  // el manejo de la barra de estado.
```

```
// En este caso, se establece en que no debe superponerse con la app.  
await StatusBar.setOverlaysWebView({ overlay: false });  
// Y se implementa que su color debe seguir la temática de la app.  
await StatusBar.setStyle({ style: Style.Default });  
}
```

```
// Los safe-areas son unas variables que entrega  
// el sistema operativo a la aplicación.  
// Estas variables representan las zonas que ocupan  
// partes nativas del sistema  
(barra de estado o botonera inferior del sistema nativo).  
// De este modo, añadiendo padding en los cuatro lados  
// en estilos globales del desarrollo,  
// la aplicación se adaptará al tamaño de forma automática  
// sin solapes, independientemente del menú que aparezca.  
body {  
  padding-top: env(safe-area-inset-top);  
  padding-bottom: env(safe-area-inset-bottom);  
  padding-left: env(safe-area-inset-left);  
  padding-right: env(safe-area-inset-right);  
}
```

Apertura de PDFs en Android (FileSystem + FileOpener)

En PC, incrustar un PDF con `iframe` suele funcionar bien, pero en Android el WebView puede dar problemas. La solución aplicada consiste en:

1. Detectar que se está en móvil.
2. Mostrar un botón *Abrir PDF*.
3. Guardar temporalmente el archivo en el dispositivo.
4. Abrirlo con el visor nativo del sistema.

Un fragmento de código representativo es el que muestra el caso de los detalles de cada rama de la enseñanza, en el cual, puede haber un libro del curso. Este libro, proviene de la base de datos y aparece en pantalla (todo esto tratado en el proyecto original). La evolución a móvil impide la vista directamente ya que no hay un visor de PDF integrado en la app (lo cual sí que aparece en los navegadores), por lo tanto, es necesario abrirlo vía externa en el visor del sistema accediendo así a las funcionalidades nativas mediante los plugins de Capacitor, en concreto, se usarán FileSystem y FileOpener.

```
//Se comprueba al inicio si es pantalla reducida
(sistemas Android o iOS de tablet y móvil).
@HostListener('window:resize')
onResize() {
  this.checkIsMobile();
}
private checkIsMobile(): void {
  this.isMobile = window.matchMedia('(max-width: 1368px)').matches;
}
// Un botón solo accesible desde móvil acciona el método.
async openPdf(): Promise<void> {
  try {
    if (!this.libroBlob) {
      await this
        .presentToast('No hay PDF disponible para abrir.', 'danger');
      return;
    }
    // En caso de haber libro, es necesario obtenerlo en base64
    // para poder construirlo fuera del dominio de la aplicación
    const base64 = await this.blobToBase64(this.libroBlob);
    const fileName =
      `libro-apoyo-${this.ramaConfig?._id || 'rama'}.pdf`;
    // El plugin FileSystem construye el documento a partir
    // del código en base64 y lo guarda en memoria
    // generando así una URL real y accesible.
    const result = await FileSystem.writeFile({
      path: fileName,
      data: base64,
      directory: Directory.Cache, // Guardado temporal
      recursive: true,
    });
    // El plugin FileOpener le indica que tipo de formato tiene
    // el archivo para que lo abra con la aplicación predeterminada
    // del sistema (en este caso PDF) y su URL para ubicarlo y abrirlo.
    await FileOpener.open({
      filePath: result.uri,
      contentType: 'application/pdf',
      openWithDefault: true,
    });
  } catch (e) {
    console.error(e);
    await this.presentToast
      ('No se pudo abrir el PDF en este dispositivo.', 'danger');
  }
}
```



```
}
```

Estilos específicos para móvil

Además de los cambios anteriores, se han aplicado ajustes menores de scss para algunas partes de la aplicación que, gracias a Ionic combinado con Capacitor, son muy ligeros. El ejemplo más elaborado es el cambio en el menú de botones que le aparecía al profesor en la lista de tareas creadas de cada rama, en este caso, el menú de botones provocaba que en pantallas pequeñas no se llegara a ver, por tanto, se detecta el tamaño de pantalla que es y, en caso de tamaño pequeño manejado con @media, se pasa a un solo botón que ya, pulsándolo, despliega el resto de opciones.

```
// En caso de tener un tamaño demasiado pequeño, esconde
// los tres botones y los encapsula en un único botón desplegable.
@media (max-width: 767px) {
  .desktop-only {
    display: none;
  }

  .mobile-only {
    display: flex;
  }
}
// En caso contrario, los muestra y esconde el botón único.
.desktop-only {
  display: flex;
}

.mobile-only {
  display: none;
}
```

Ajustes finales en Android Studio (Iconos, Pantallas de carga y XML)

Una vez compilado el proyecto Android, ciertos recursos visuales se terminan de ajustar desde **Android Studio**, como:

- Iconos de la aplicación en vistas nativas.
- Pantalla de carga.

Todo esto lo maneja Android Studio para cada aplicación añadiendo unos XML que contienen las referencias a los distintos logos y estilos usados, en concreto, AndroidManifest (donde se declara el nombre, icono y permisos) y splash.xml (pantalla de carga)

6.2 DISEÑO Y EXPERIENCIA DE USUARIO (UX/UI)

El diseño del frontend en la solución extendida se apoya directamente en la interfaz y los principios UX/UI definidos en la primera versión, descritos en el apartado correspondiente. En este complemento no se plantea un rediseño completo de la aplicación, sino una **adaptación visual y funcional al contexto móvil**, derivada de la exportación a Android mediante Capacitor y de los ajustes descritos en la sección 6.1.

De este modo, el objetivo principal de esta sección es mostrar **cómo los cambios implementados se traducen en mejoras perceptibles para el usuario final**, garantizando una experiencia coherente, estable y visualmente consistente cuando la aplicación se ejecuta como app instalada.

6.2.1 Continuidad visual y coherencia de interacción

Tal y como se ha detallado en el apartado de implementación, la aplicación mantiene intacta la estructura de navegación, los patrones de interacción y la identidad visual definidos en el proyecto base. Esto permite que la experiencia de usuario sea continua entre la versión web y la versión móvil, evitando una curva de aprendizaje adicional.

En particular, gracias al uso de Ionic y a la correcta integración con Capacitor:

- Se conserva la **organización funcional de pantallas**, manteniendo la separación entre navegación académica y navegación de gestión, como se aprecia en la vista principal mostrada en la figura 6.1.
- Se mantienen los **patrones de interacción habituales**, como el uso de modales, listas dinámicas y formularios guiados garantizando una experiencia predecible y consistente.
- La aplicación presenta una **identidad gráfica homogénea** independientemente del dispositivo, reforzando la percepción de producto unificado.

6.2.2 Efectos visibles de la adaptación a móvil

Tal y como se ha expuesto en la sección 6.1, la exportación a Android ha requerido una serie de ajustes técnicos específicos. Desde el punto de vista del usuario, estas adaptaciones se traducen en mejoras directas en la experiencia visual y funcional:

- **Ausencia de solapes con elementos del sistema:** Gracias al uso de *safe areas* y a la configuración del plugin *StatusBar*, la interfaz se adapta automáticamente a la presencia de la barra superior y la navegación inferior del sistema, garantizando una correcta visualización del contenido, como se observa en la figura 6.2.
- **Gestión nativa de documentos PDF:** Los documentos PDF se abren mediante el visor nativo del sistema, proporcionando una lectura fluida y estable.

- **Reorganización adaptativa de controles:** En pantallas reducidas, algunos menús de acciones se condensan en botones desplegados, preservando la accesibilidad sin comprometer la claridad visual.
- **Ajustes de tamaño y posición:** Algunos elementos, al basarse en el porcentaje ocupado de la página para posicionarse y ocupar un tamaño, no ofrecían una buena experiencia de usuario en pantallas pequeñas, por tanto, también eso ha requerido de ajustes como se puede ver en la comparativa del tamaño del vídeo en los cuestionarios (tanto en edición como en creación y entrega) entre la versión web y la versión Android en la figura 6.3.
- **Percepción de aplicación instalada:** La configuración de iconos y pantallas de carga refuerza la sensación de aplicación nativa, como se aprecia en el logo de la aplicación representada en la figura 6.4.

6.2.3 Ejemplos visuales de la solución extendida

Las siguientes figuras ilustran los principales efectos visuales derivados de la implementación descrita previamente, mostrando la ejecución de la aplicación como app instalada en Android, la correcta adaptación al espacio disponible, la apertura de documentos PDF mediante el visor nativo y la personalización de los recursos visuales.

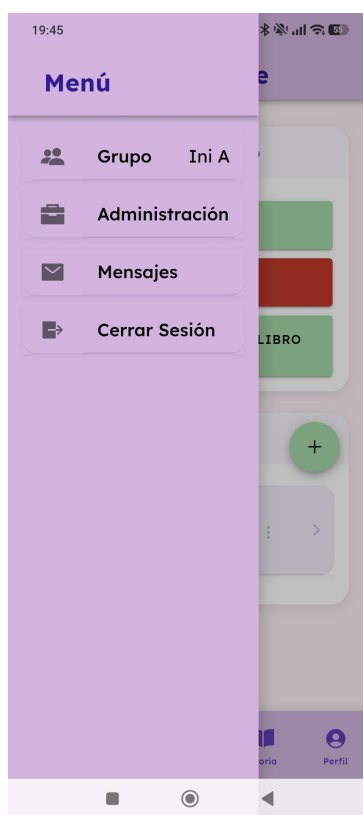


Figura 6.1: Menú doble (lateral e inferior)

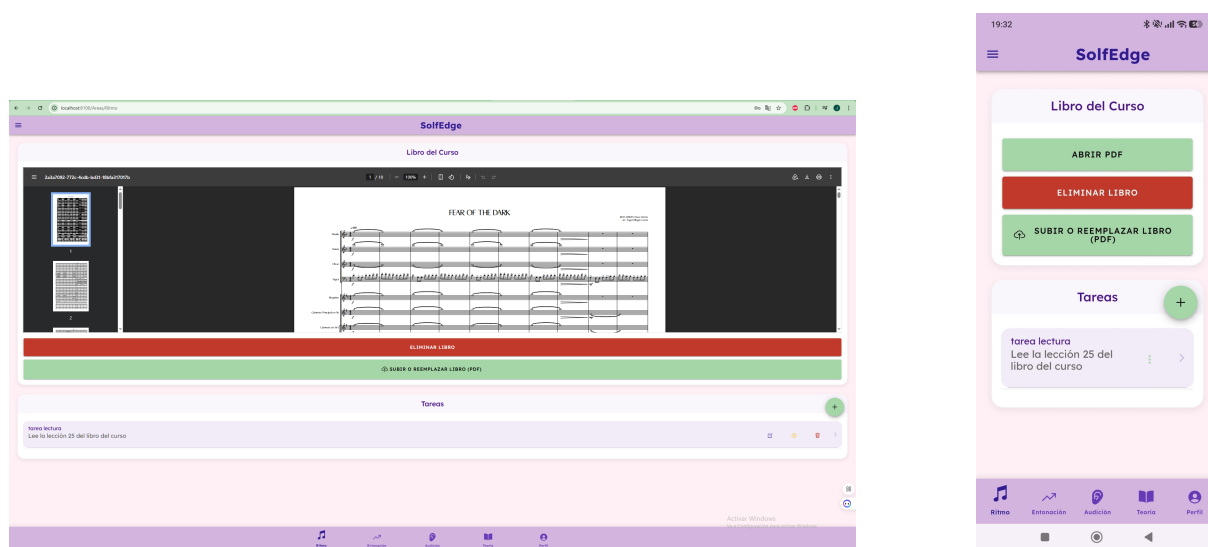


Figura 6.2: Vista de la rama de ritmo en versión web (izquierda) y Android (derecha).

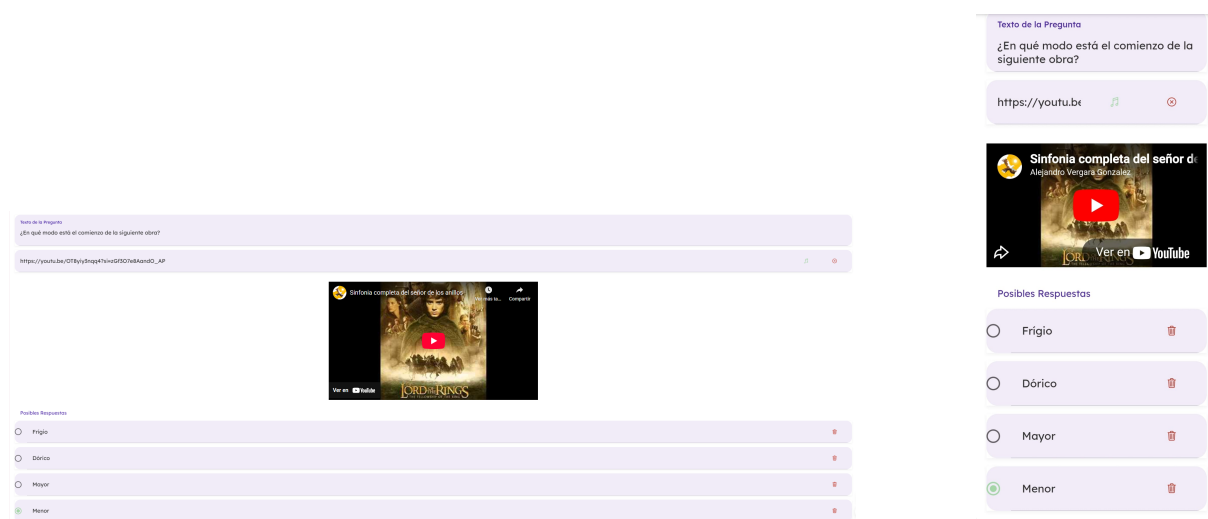


Figura 6.3: Vista de cuestionario en versión web (izquierda) y Android (derecha).

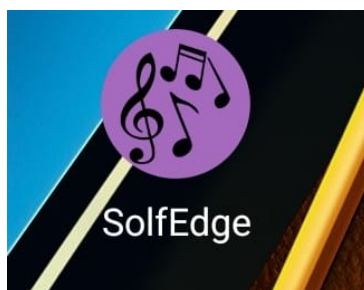


Figura 6.4: Icono desde vista nativa

En conjunto, estos elementos reflejan cómo la implementación descrita previamente se traduce en una experiencia de usuario consistente, fluida y adaptada al entorno móvil, preservando la identidad visual y funcional del proyecto original.

6.3 TESTING

El frontend mantiene el enfoque de testing definido en la versión inicial, utilizando **Jasmine** como framework de pruebas unitarias y **Karma** como entorno de ejecución. Como se expuso en el trabajo previo, esta combinación permite validar de forma sistemática la lógica interna de los componentes Angular y su correcta integración con los servicios y la interfaz de usuario.

Sobre esta base ya establecida, la solución extendida no introduce cambios estructurales en la metodología de pruebas, sino que amplía el conjunto de tests existentes para cubrir las nuevas funcionalidades incorporadas. En particular, se ha reforzado el testing en aquellos componentes que presentan una mayor complejidad lógica o que introducen flujos críticos de interacción, especialmente aquellos relacionados con la gestión administrativa del sistema.

En este contexto, se ha desarrollado un conjunto específico de pruebas para el panel de gestión de profesores, accesible por el administrador, cuyo objetivo es permitir la revisión y eliminación controlada de usuarios con rol docente tal y como se establece en la definición del proyecto. Este componente introduce nuevas posibilidades que requieren ser verificadas mediante tests automatizados, tales como la confirmación explícita de acciones destructivas, la validación de identificadores y la correcta gestión de errores.

De manera concreta, los tests implementados persiguen los siguientes objetivos:

- Verificar la correcta inicialización del componente y la carga inicial del listado de profesores desde el servicio backend.
- Comprobar la correcta interacción con los servicios de negocio mediante la simulación controlada de respuestas.
- Validar los flujos de interacción que requieren confirmación explícita por parte del usuario antes de ejecutar acciones irreversibles.
- Garantizar la actualización coherente del estado interno del componente tras operaciones de eliminación.
- Verificar el tratamiento de situaciones de error y su correcta notificación visual mediante alertas y mensajes emergentes.

Para ello, se emplea el mismo enfoque de aislamiento descrito en el proyecto base, utilizando *spies* de Jasmine sobre los controladores de Ionic y los servicios de negocio. Este mecanismo permite simular de modo controlado el comportamiento de dependencias externas, evitando efectos colaterales durante

la ejecución de las pruebas. Asimismo, se utilizan herramientas específicas para el control del flujo asíncrono, lo que permite reproducir con precisión operaciones basadas en promesas y observables.

A continuación se presenta un pseudocódigo estructurado que resume el comportamiento validado por los tests implementados para el componente de gestión de profesores. Se ha decidido usar esta representación porque mantiene un nivel de detalle cercano a la semántica real facilitando su comprensión y la implementación real resulta demasiado extensa y depende de detalles técnicos

```
Inicializar entorno de pruebas

Crear mocks de dependencias:
- ModalController
- ToastController
- AlertController
- UsuarioService

Crear spies para métodos críticos:
- dismiss()
- create() y present() de alertas y toasts
- getAllProfesores()
- deleteUsuario()

Configurar TestBed con el componente y los mocks
Crear instancia del componente
Ejecutar ciclo inicial (ngOnInit)

TEST 1: Creación del componente
  Verificar que la instancia se crea correctamente

TEST 2: Carga inicial de profesores
  Simular respuesta del servicio
  Ejecutar ngOnInit
  Verificar llamada al servicio y asignación interna

TEST 3: Cierre del modal
  Ejecutar dismissModal
  Verificar invocación del ModalController

TEST 4: Solicitud de eliminación
  Ejecutar deleteProfesor(id)
  Verificar aparición de alerta de confirmación

TEST 5: ID inválido
  Ejecutar deleteProfesor('')
  Verificar aparición de toast de error
```

Comprobar que no se invoca deleteUsuario

TEST 6: Confirmación de eliminación

Precargar lista interna

Confirmar eliminación

Verificar:

- llamada al servicio
- aparición de toast de éxito
- actualización correcta del listado

TEST 7: Error en eliminación

Simular error del servicio

Confirmar eliminación

Verificar:

- aparición de toast de error
- no modificación del estado interno

TEST 8: Cancelación

Abrir alerta

No confirmar eliminación

Verificar que no se invoca deleteUsuario

Este conjunto de pruebas permite validar sistemáticamente los principales flujos funcionales introducidos en la solución extendida, garantizando un comportamiento estable ante escenarios normales y situaciones de error. De este modo, se refuerza la fiabilidad global del frontend y se reduce el riesgo de regresiones durante la evolución futura del sistema.

7 DESPLIEGUE DEL SISTEMA

El despliegue del sistema mantiene la misma infraestructura y metodología establecidas en el proyecto original, basadas en un flujo de integración y entrega continua que garantiza la estabilidad, trazabilidad y fiabilidad del software. En este complemento, el objetivo no es redefinir dicho proceso, sino extenderlo con un nuevo escenario de despliegue: la generación de una aplicación móvil Android instalada.

De este modo, el despliegue web se conserva sin modificaciones relevantes, mientras que la principal aportación de esta fase se centra en la incorporación de un flujo específico para la construcción, empaquetado y validación de la versión móvil, apoyado en el uso de Capacitor y Android Studio, tal y como se ha descrito en la sección 6.1.

7.1 DESPLIEGUE WEB

Tal y como se detalla en el documento principal, el sistema se despliega mediante un flujo de integración y entrega continua basado en **GitHub Actions**, con publicación automática del backend en **Render** y del frontend web en **GitHub Pages**, apoyándose en una base de datos gestionada en la nube mediante **MongoDB Atlas**.

Este enfoque permite automatizar la ejecución de pruebas, la construcción de artefactos y la publicación de nuevas versiones, garantizando estabilidad operativa, control de versiones y detección temprana de errores. Dado que este mecanismo no ha sufrido modificaciones en el presente complemento, no se reproduce aquí su descripción completa, remitiendo al lector al capítulo correspondiente del proyecto original.

7.2 DESPLIEGUE MÓVIL: GENERACIÓN DE LA APLICACIÓN ANDROID

La principal novedad introducida en esta fase es la posibilidad de ejecutar la aplicación como una **app instalada en dispositivos Android**, partiendo directamente del frontend híbrido desarrollado con Ionic y Angular. Esta ampliación responde al objetivo de reforzar la portabilidad del sistema y facilitar su uso en contextos educativos reales, donde el acceso desde dispositivos móviles resulta especialmente relevante.

El proceso de despliegue móvil se apoya en dos herramientas principales:

- **Capacitor**, como puente entre la aplicación web y el entorno nativo móvil.
- **Android Studio**, como entorno oficial para la construcción, depuración y empaquetado de la aplicación Android.

7.2.1 Generación del contenedor Android mediante Capacitor

Capacitor permite encapsular la aplicación web en un contenedor nativo Android, proporcionando una capa de comunicación entre el código JavaScript y las funcionalidades propias del sistema operativo. Este mecanismo hace posible que la aplicación híbrida se comporte como una app instalada, manteniendo una única base de código para las versiones web y móvil.

El flujo general seguido para la generación del proyecto Android es el siguiente:

1. Compilación del frontend Angular en modo producción.
2. Sincronización del proyecto web con Capacitor.
3. Generación automática de la estructura Android nativa.
4. Apertura del proyecto resultante en Android Studio.

Este procedimiento permite reutilizar íntegramente la lógica y la interfaz desarrolladas, evitando duplicidades y manteniendo la coherencia funcional entre plataformas.

7.2.2 Construcción y validación en Android Studio

Una vez generado el proyecto Android, Android Studio se utiliza como entorno principal para:

- Gestionar dependencias nativas.
- Compilar el contenedor Android junto con el WebView.
- Ejecutar la aplicación en emuladores y dispositivos físicos.
- Depurar errores específicos de plataforma.
- Generar el paquete instalable (APK).

Este entorno permite además realizar los ajustes finales en recursos gráficos, configuración del manifiesto, permisos y pantallas de carga, tal y como se ha descrito en la sección 6.1. De este modo, la aplicación adquiere una apariencia plenamente nativa, reforzando la percepción de producto final estable y profesional.

7.2.3 Generación del paquete instalable

El proceso culmina con la construcción de una **APK**, que permite instalar la aplicación directamente en dispositivos Android sin necesidad de pasar por tiendas de aplicaciones. Este paquete constituye un artefacto verificable que materializa la portabilidad efectiva del sistema, permitiendo evaluar su comportamiento en condiciones reales de uso.

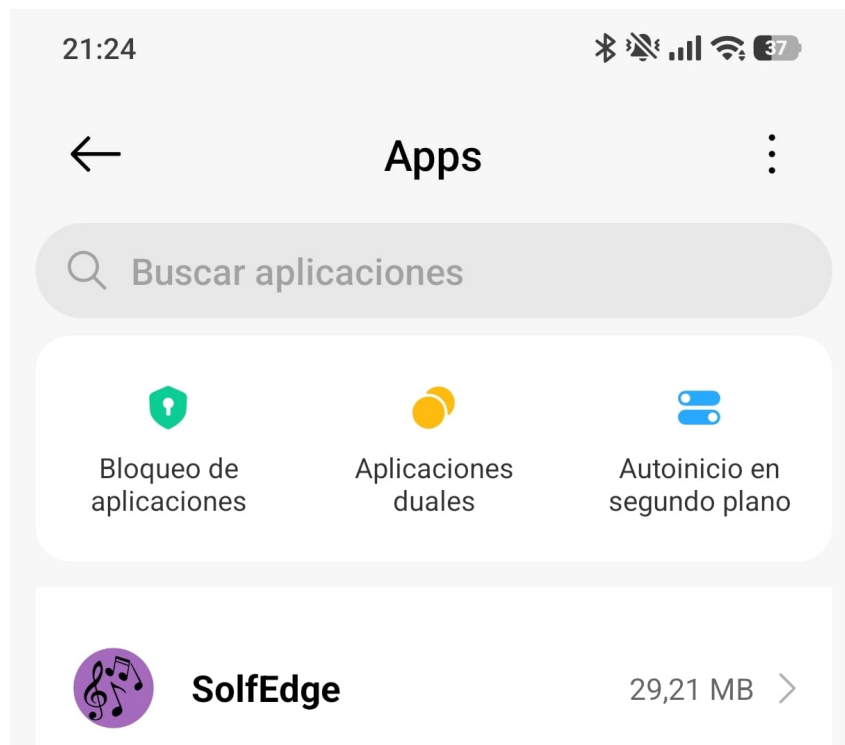


Figura 7.1: Aplicación instalada en un dispositivo Android

7.3 VALOR AÑADIDO DEL DESPLIEGUE MÓVIL

La incorporación del despliegue móvil aporta un valor añadido significativo al sistema, ya que:

- Refuerza la accesibilidad para alumnado y profesorado.
- Facilita el uso en entornos educativos reales.
- Consolida la arquitectura híbrida adoptada.
- Permite futuras extensiones a otras plataformas móviles sin cambios estructurales.

De este modo, el despliegue móvil no se concibe como una mera adaptación técnica, sino como un paso estratégico hacia la implantación real del sistema, alineado con los objetivos definidos en el capítulo 2.

8 PLAN DE IMPLANTACIÓN

El presente capítulo describe un plan integral para la implantación progresiva de la solución desarrollada en un entorno educativo real. Aunque durante el desarrollo de este proyecto no se ha llevado a cabo una implantación completa siguiendo dicho plan, sí se ha realizado una validación preliminar en un centro educativo, mostrando el funcionamiento general de la aplicación y recogiendo observaciones directas del profesorado. Esta experiencia, descrita en el apartado 8.11, ha permitido detectar mejoras relevantes y refuerza la necesidad de contar con un plan estructurado que guíe una futura implantación realista.

El objetivo de este plan no es documentar un proceso ya ejecutado, sino establecer un marco estratégico, técnico y organizativo que permita, en un escenario real, desplegar la aplicación de forma segura, estable, escalable y sostenible, contemplando tanto los aspectos tecnológicos como los humanos y organizativos implicados. Este enfoque resulta coherente con la evolución funcional y tecnológica presentada en los capítulos 5 y 6.

Además, cabe destacar que las decisiones técnicas adoptadas a lo largo del proyecto han favorecido de manera significativa su adaptación a un contexto real de implantación. Las tecnologías seleccionadas, aunque en su configuración actual priorizan el uso de recursos gratuitos y presentan limitaciones para un entorno productivo, ofrecen planes de escalado superiores o permiten una migración sencilla hacia infraestructuras de pago, sin necesidad de reestructurar la arquitectura del sistema. Asimismo, aunque se han identificado determinados aspectos susceptibles de mejora en materia de seguridad, estos quedan documentados como líneas de evolución futura, manteniendo la versión actual del sistema suficientemente preparada para asumir dichos cambios de forma progresiva y controlada.

8.1 OBJETIVOS DEL PLAN DE IMPLANTACIÓN

La implantación de un sistema informático en un entorno educativo requiere una planificación cuidadosa que tenga en cuenta múltiples factores. En este contexto, el plan de implantación persigue los siguientes objetivos principales:

- Garantizar una transición progresiva desde el entorno de desarrollo hacia un entorno productivo real, minimizando interrupciones en la actividad docente.
- Asegurar la estabilidad técnica del sistema, su disponibilidad continua y la integridad de los datos almacenados, en coherencia con los principios arquitectónicos definidos en el apartado 3.7 y en la versión inicial.

- Facilitar la adopción de la herramienta por parte del profesorado y el alumnado, reduciendo la resistencia al cambio mediante interfaces intuitivas y procesos alineados con la práctica educativa real.
- Proteger los datos personales gestionados por el sistema, especialmente aquellos relativos a menores, cumpliendo con la normativa vigente en materia de protección de datos.
- Definir un marco de mantenimiento, soporte y evolución que garantice la sostenibilidad del proyecto a medio y largo plazo.

Estos objetivos sitúan la implantación como una fase crítica dentro del ciclo de vida del sistema, en la que convergen aspectos técnicos, organizativos, legales y pedagógicos, complementando el proceso de desarrollo descrito en los capítulos anteriores.

8.2 CONTEXTO DE IMPLANTACIÓN EN CENTROS EDUCATIVOS

La aplicación ha sido concebida para su uso en centros educativos dedicados a la enseñanza musical, tales como conservatorios profesionales, escuelas municipales de música o academias privadas. Este enfoque se deriva directamente del análisis funcional y pedagógico realizado en el proyecto original y sintetizado en el capítulo 1.

Estos entornos presentan una serie de características comunes que condicionan el proceso de implantación:

- Existencia de múltiples perfiles de usuario con roles claramente diferenciados, tal y como se definió en la especificación funcional del sistema (apartado 3.2).
- Gestión simultánea de múltiples grupos, niveles académicos y asignaturas.
- Necesidad de acceso desde distintos dispositivos y ubicaciones, lo que refuerza la importancia de la portabilidad móvil abordada en el apartado 6.1.
- Infraestructuras tecnológicas heterogéneas y, en muchos casos, limitadas.

Además, estos centros suelen contar con sistemas propios de gestión académica, lo que introduce la necesidad de contemplar procesos de migración e integración de datos. En este contexto, la implantación del sistema debe realizarse de forma progresiva y flexible, adaptándose a las particularidades organizativas y tecnológicas de cada institución.

8.3 ESTRATEGIA DE IMPLANTACIÓN PROGRESIVA

La implantación del sistema se estructura como un proceso progresivo, planificado en fases claramente diferenciadas, con el objetivo de reducir riesgos técnicos y organizativos, facilitar la adaptación de los usuarios y garantizar la estabilidad del servicio antes de su adopción completa.

La implantación se ejecutará en tres fases sucesivas, cada una con objetivos, tareas y criterios de validación claramente definidos.

8.3.1 Fase 1: Despliegue piloto

Durante esta primera fase, el sistema se implantará en un entorno real pero controlado, con un número reducido de usuarios. Esta fase se desarrollará durante un periodo estimado de **cuatro a seis semanas**.

El despliegue piloto se realizará con:

- Un único profesor voluntario.
- Uno o dos grupos reducidos de alumnos.

Las acciones concretas que se llevarán a cabo son:

1. Despliegue del backend y frontend en el entorno productivo seleccionado.
2. Configuración inicial del sistema y validación de la infraestructura.
3. Creación controlada de usuarios reales y verificación de accesos por rol.
4. Importación de datos académicos mediante los mecanismos desarrollados.
5. Uso cotidiano del sistema en actividades docentes reales, siguiendo la operativa definida en los casos de uso y requisitos funcionales descritos en el documento.
6. Registro sistemático de incidencias técnicas y funcionales, así como recopilación de feedback del profesorado.

Los objetivos verificables de esta fase son:

- Validar el correcto funcionamiento del sistema bajo condiciones reales.
- Identificar errores no detectados en el entorno de desarrollo.
- Evaluar la estabilidad general de la aplicación (backend, frontend y comunicaciones).
- Recoger feedback directo del profesorado y alumnado para priorizar ajustes.

Al finalizar esta fase se elaborará un **informe técnico de validación**, que servirá como base para introducir ajustes correctivos antes de avanzar a la siguiente etapa.

8.3.2 Fase 2: Despliegue parcial

Una vez validados los resultados del piloto, se procederá a un despliegue parcial ampliado durante un periodo aproximado de **dos a tres meses**.

En esta fase, el sistema se extenderá a:

- Varios profesores.
- Múltiples grupos de distintos niveles dentro del mismo centro.

Las acciones planificadas son:

1. Ampliación progresiva del número de usuarios y grupos activos.
2. Verificación de rendimiento bajo mayor carga (usuarios concurrentes, accesos a materiales, entregas y cuestionarios).
3. Ajuste de los procesos administrativos internos del centro relacionados con altas/bajas y gestión de grupos.
4. Monitorización intensiva del consumo de recursos y revisión de logs para detectar cuellos de botella.
5. Consolidación de procedimientos de soporte: canal de incidencias, prioridades y tiempos de respuesta.

Los objetivos principales son:

- Validar la escalabilidad del sistema.
- Detectar cuellos de botella de rendimiento o límites de infraestructura.
- Refinar la gestión de recursos multimedia y archivos.
- Ajustar los flujos reales de trabajo del profesorado, reduciendo fricciones.

8.3.3 Fase 3: Despliegue completo

Tras completar con éxito la fase parcial, se ejecutará el despliegue completo en el centro educativo, integrando el sistema como herramienta habitual.

Las acciones serán:

1. Alta y verificación de todos los profesores participantes.
2. Migración completa de grupos y datos académicos.

3. Activación del servicio como sistema operativo estándar del centro para las funcionalidades definidas.
4. Activación completa de los mecanismos de monitorización y respaldo.

Esta fase marca la transición del sistema a **herramienta central de apoyo docente**, integrándose en la operativa diaria del centro.

8.4 MIGRACIÓN TECNOLÓGICA HACIA UN ENTORNO PRODUCTIVO

La migración tecnológica constituye un eje central del plan de implantación, cuyo objetivo es transformar la infraestructura académica y experimental en una plataforma productiva, escalable y robusta. Este proceso se apoya en la arquitectura modular descrita en el apartado 3.7, que permite sustituir piezas de infraestructura minimizando el impacto sobre el código.

8.4.1 Justificación de la migración

Durante el desarrollo del proyecto se priorizó el uso de tecnologías gratuitas por motivos académicos. En un escenario real de operación continuada, esta infraestructura puede resultar limitada en términos de disponibilidad, escalado, monitorización y soporte. Por tanto, se realizará una migración controlada hacia servicios cloud profesionales que permitan:

- Escalado automático bajo picos de carga.
- Mayor disponibilidad y redundancia.
- Monitorización avanzada y alertas.
- Soporte técnico profesional y SLAs.

8.4.2 Migración del backend y frontend a Microsoft Azure

La migración de los servicios web hacia un entorno productivo se apoyará en la plataforma **Microsoft Azure** [5], aprovechando su ecosistema cloud maduro, su integración con herramientas de desarrollo modernas y su capacidad para proporcionar una infraestructura altamente escalable, resiliente y monitorizable.

La arquitectura definida en el apartado 3.7 facilita este proceso, ya que el sistema presenta una clara separación entre frontend, backend y base de datos, permitiendo una transición progresiva sin necesidad de modificar la lógica de la aplicación. En este contexto, Azure App Service [6] se utilizará como plataforma unificada tanto para el despliegue del backend como del frontend, garantizando una gestión homogénea de los recursos y un control centralizado del entorno productivo.

Azure App Service como plataforma de ejecución

Azure App Service proporciona un entorno gestionado para la ejecución de aplicaciones web, APIs REST y servicios backend, abstraendo la complejidad asociada a la gestión directa de servidores, redes, balanceo de carga y sistemas operativos. Este modelo permite centrar los esfuerzos en el desarrollo funcional, delegando en la plataforma cloud la responsabilidad del escalado, la disponibilidad y la seguridad perimetral.

Desde el punto de vista técnico, Azure App Service ofrece las siguientes capacidades clave:

- **Escalado vertical:** Permite aumentar dinámicamente la capacidad de cómputo asignada a la aplicación (CPU, memoria RAM y rendimiento de disco), adaptando el entorno de ejecución a mayores cargas sin modificar la arquitectura del sistema.
- **Escalado horizontal:** Posibilita la ejecución simultánea de múltiples instancias de la aplicación, distribuyendo la carga de peticiones entrantes mediante balanceo automático. Este mecanismo resulta especialmente relevante en escenarios con alta concurrencia, ya que permite absorber incrementos súbitos de tráfico manteniendo tiempos de respuesta estables.
- **Gestión automática de infraestructura:** Azure se encarga del aprovisionamiento, monitorización y recuperación ante fallos de las instancias, garantizando una elevada disponibilidad del servicio sin intervención manual.
- **Integración nativa con CI/CD:** Azure App Service se integra de forma directa con GitHub Actions, facilitando la automatización completa del flujo de despliegue, tal y como se describe en el capítulo 7. En consecuencia, todas las operaciones de la migración quedarán integradas de forma automática con tan solo unas líneas en el archivo `.YML` que lee GitHub Actions.

Este conjunto de características convierte a Azure App Service en una plataforma idónea para albergar tanto el backend Node.js como el frontend Angular/Ionic, permitiendo un despliegue coherente, altamente disponible y preparado para un crecimiento progresivo del número de usuarios.

Arquitectura de despliegue propuesta

La arquitectura de despliegue en Azure se estructura en los siguientes componentes principales:

- **Backend:** Desplegado en un recurso Azure App Service configurado para Node.js, con políticas de escalado automático basadas en consumo de CPU, memoria y volumen de peticiones HTTP.
- **Frontend:** Desplegado también sobre Azure App Service, sirviendo los recursos estáticos generados por el proceso de construcción de Angular/Ionic, permitiendo igualmente escalado horizontal bajo alta carga.
- **Servicios auxiliares:** Almacenamiento de variables de entorno, gestión segura de secretos y configuración centralizada mediante Azure Key Vault [7].

Esta disposición garantiza un comportamiento homogéneo en ambos extremos de la aplicación y facilita la administración global del sistema desde un único entorno cloud.

Proceso de migración

La migración de los servicios web se ejecutará siguiendo un proceso estructurado en cuatro fases claramente definidas:

1. **Creación de infraestructura base en Azure:** Provisión de recursos App Service para backend y frontend, definición de grupos de recursos, configuración de redes virtuales, reglas de firewall y establecimiento de variables de entorno seguras.
2. **Configuración de despliegue automatizado:** Adaptación del pipeline de integración y despliegue continuo descrito en el proyecto original, modificando exclusivamente los endpoints de despliegue para apuntar a Azure, manteniendo intacta la lógica de CI/CD.
3. **Despliegue inicial en Azure y validación funcional:** Verificación exhaustiva del correcto funcionamiento del sistema, comprobando procesos de autenticación, acceso a la base de datos, ejecución de operaciones CRUD principales y respuesta bajo carga simulada.
4. **Cambio controlado de tráfico:** Una vez validado el entorno productivo, redirección progresiva del tráfico desde el entorno anterior hacia Azure, minimizando interrupciones del servicio.

Este procedimiento permite garantizar una transición segura, reversible y con impacto mínimo sobre los usuarios finales.

Pipeline de despliegue automatizado

El objetivo principal de esta migración es que el cambio afecte únicamente al flujo de despliegue, sin introducir modificaciones en la lógica del sistema. La estructura modular del proyecto y el uso de contenedores Docker [8] permiten desacoplar completamente la ejecución de la aplicación del entorno subyacente, lo que facilita la transición desde un despliegue local hacia una infraestructura en la nube sin necesidad de reescritura del código.

La aplicación desarrollada, la cual será definida por primera vez como *SolfEdge*, se despliega en la plataforma Microsoft Azure utilizando el servicio *Azure App Service for Containers*, que permite ejecutar contenedores Docker de forma gestionada, escalable y altamente disponible. Como registro privado se emplea *Azure Container Registry* (ACR) [9], que actúa como repositorio centralizado de imágenes Docker.

Para la correcta ejecución del pipeline de despliegue es necesario realizar previamente una serie de configuraciones en Azure, que se detallan a continuación.

Configuración previa en Azure Antes de poder automatizar el despliegue, es imprescindible preparar la infraestructura mínima necesaria en la plataforma Azure:

- **Creación del grupo de recursos:** Se define un grupo de recursos (por ejemplo, RCPMA) como convención de nomenclatura para agrupar todos los servicios asociados. Este identificador se emplea a modo ilustrativo y puede adaptarse al centro que realice la implantación.

- **Creación del Azure Container Registry (ACR):** Se crea un registro privado de contenedores donde se almacenarán las imágenes Docker del frontend y backend de SolfEdge. Este registro permite la gestión segura, versionada y centralizada de las imágenes, facilitando su despliegue automatizado.
- **Creación de las Web Apps para contenedores:** Se crean dos instancias independientes de *Azure App Service for Containers* (sobre sistema Linux), una destinada al frontend (*solfedge-frontend*) y otra al backend (*solfedge-backend*). Estas aplicaciones se configuran para obtener sus imágenes desde el ACR.
- **Creación del Service Principal:** Se genera una identidad de servicio con permisos suficientes para autenticarse contra Azure, subir imágenes al ACR y modificar la configuración de las Web Apps. Las credenciales asociadas a este Service Principal se almacenan como secretos seguros en GitHub Actions.

Esta configuración inicial permite disponer de un entorno completamente preparado para recibir despliegues automáticos desde el repositorio de código.

Flujo general de despliegue El proceso completo de despliegue automatizado se integra dentro del pipeline CI/CD del proyecto, ejecutándose de forma automática cada vez que se realiza un *push* sobre la rama principal del repositorio. Dicho flujo consta de las siguientes fases:

1. Descarga del código fuente desde el repositorio.
2. Autenticación segura contra Azure mediante credenciales almacenadas en GitHub Secrets.
3. Construcción de las imágenes Docker correspondientes al frontend y backend.
4. Publicación de dichas imágenes en el Azure Container Registry.
5. Actualización automática de las Web Apps para que utilicen la nueva versión de las imágenes.

Este enfoque garantiza un despliegue reproducible, trazable y consistente, eliminando la intervención manual y reduciendo drásticamente la probabilidad de errores humanos.

Pipeline de despliegue A continuación se muestra un pipeline orientativo que ilustra el proceso completo de construcción, publicación y despliegue automático de SolfEdge en Azure. Este pipeline se apoya en las fases previas de construcción y testeo ya descritas, adaptando únicamente la etapa final de despliegue al entorno cloud.

```
push-and-deploy-frontend:
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
```



```
needs: build-frontend

steps:
  - name: Checkout code
    uses: actions/checkout@v3

  - name: Login to Azure CLI
    uses: azure/login@v1
    with:
      creds: ${ secrets.AZURE_CREDENTIALS }

  - name: Download artifacts
    uses: actions/download-artifact@v4
    with:
      name: www
      path: ./www

  - name: Build and push frontend Docker image to ACR
    run: |
      IMAGE_NAME=repodocker.azurecr.io/solfedge-frontend:
      ${ github.run_number }
      docker build -t $IMAGE_NAME ./frontend
      echo ${ secrets.AZURE_PASSWORD } | docker login \
      repodocker.azurecr.io -u ${ secrets.AZURE_USERNAME } \
      --password-stdin
      docker push $IMAGE_NAME

  - name: Deploy frontend to Azure Web App for Containers
    run: |
      IMAGE_NAME=repodocker.azurecr.io/solfedge-frontend:
      ${ github.run_number }
      az webapp config container set \
      --docker-custom-image-name $IMAGE_NAME \
      --docker-registry-server-password ${ secrets.AZURE_PASSWORD } \
      --docker-registry-server-url https://repodocker.azurecr.io \
      --docker-registry-server-user ${ secrets.AZURE_USERNAME } \
      --name solfedge-frontend \
      --resource-group RCPMA

push-and-deploy-backend:
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  needs: build-backend

  steps:
```



```
- name: Checkout code
  uses: actions/checkout@v3

- name: Login to Azure CLI
  uses: azure/login@v1
  with:
    creds: ${{ secrets.AZURE_CREDENTIALS }}

- name: Build and push backend Docker image to ACR
  run: |
    IMAGE_NAME=repodocker.azurecr.io/solfedge-backend:
    ${{ github.run_number }}
    docker build -t $IMAGE_NAME ./backend
    echo ${{ secrets.AZURE_PASSWORD }} | docker login \
    repodocker.azurecr.io -u ${{ secrets.AZURE_USERNAME }} \
    --password-stdin
    docker push $IMAGE_NAME

- name: Deploy backend to Azure Web App for Containers
  run: |
    IMAGE_NAME=repodocker.azurecr.io/solfedge-backend:
    ${{ github.run_number }}
    az webapp config container set \
    --docker-custom-image-name $IMAGE_NAME \
    --docker-registry-server-password ${{ secrets.AZURE_PASSWORD }} \
    --docker-registry-server-url https://repodocker.azurecr.io \
    --docker-registry-server-user ${{ secrets.AZURE_USERNAME }} \
    --name solfedge-backend \
    --resource-group RCPMA
```

Este pipeline proporciona un mecanismo de despliegue completamente automatizado, garantizando que cada nueva versión validada del código sea publicada de forma inmediata y coherente en el entorno productivo. La combinación de Docker, Azure App Service y GitHub Actions permite establecer un flujo DevOps moderno, escalable y alineado con las buenas prácticas actuales de despliegue continuo.



Figura 8.1: Logo Microsoft Azure

8.4.3 Escalado de la base de datos con MongoDB Atlas

La base de datos constituye un componente crítico dentro del proceso de implantación, al centralizar toda la información académica, administrativa y de seguimiento del alumnado. Por este motivo, el plan contempla una estrategia de escalado progresivo utilizando las capacidades avanzadas de **MongoDB Atlas** [10], descritas previamente en el apartado 3.7.

Arquitectura de clúster y escalabilidad

MongoDB Atlas permite desplegar clústeres distribuidos con capacidades de:

- **Escalado vertical:** ampliación dinámica de recursos hardware (CPU, memoria y almacenamiento) asignados a cada nodo.
- **Escalado horizontal:** distribución de la carga mediante fragmentación (*sharding*), permitiendo repartir los datos y las operaciones entre múltiples nodos.
- **Alta disponibilidad:** configuración de *replica sets*, garantizando redundancia de datos y recuperación automática ante fallos de nodos individuales.

Este modelo proporciona una elevada tolerancia a fallos y un rendimiento estable incluso bajo cargas intensivas, lo que resulta especialmente relevante en escenarios con numerosos accesos concurrentes, como los que pueden darse durante periodos de evaluación o entrega masiva de tareas.

Plan de escalado progresivo

El escalado de la base de datos se ejecutará siguiendo tres medidas concretas:

1. **Migración del clúster a un plan superior:** adaptación progresiva del tamaño del clúster en función del crecimiento del número de usuarios y del volumen de operaciones concurrentes.
2. **Activación de copias de seguridad automáticas:** configuración de mecanismos de respaldo continuo y snapshots programados, garantizando la posibilidad de recuperación ante errores lógicos o fallos críticos.
3. **Configuración de alta disponibilidad:** establecimiento de réplicas redundantes y, si resulta necesario, despliegue multi-región para minimizar la latencia y aumentar la tolerancia ante incidencias geográficas.

Estas acciones permiten asegurar la continuidad del servicio, la integridad de los datos y la escalabilidad futura del sistema, alineándose con los objetivos de disponibilidad y resiliencia definidos en el apartado 8.8.

Integración con la arquitectura global

La combinación de Azure App Service para la capa de aplicación y MongoDB Atlas para la persistencia de datos da lugar a una arquitectura cloud-native altamente flexible, capaz de crecer de forma controlada sin necesidad de rediseñar la solución.

Este enfoque refuerza la idoneidad de las decisiones tecnológicas adoptadas durante el desarrollo, demostrando que, aunque se haya trabajado inicialmente con infraestructuras gratuitas, la transición hacia un entorno productivo puede realizarse mediante ajustes acotados y perfectamente planificados, preservando la coherencia del sistema y su estabilidad operativa.

8.5 PUBLICACIÓN Y DISTRIBUCIÓN OFICIAL DE LA APLICACIÓN MÓVIL

Aunque en el marco de este proyecto se ha validado técnicamente la exportación de la aplicación a Android mediante la generación de un paquete instalable (APK), la implantación real del sistema en un entorno educativo requiere un mecanismo de distribución más profesional, estable y escalable. En este contexto, la publicación oficial de la aplicación en **Google Play Store** se considera un paso clave dentro del proceso de implantación global.

Este apartado no describe un proceso ejecutado durante el desarrollo del proyecto, sino que define de manera estructurada los pasos necesarios para llevar a cabo dicha publicación como parte del despliegue real del sistema en un centro educativo, alineándose con la estrategia de implantación progresiva descrita en la Sección 8.3.

8.5.1 Justificación de la publicación en tienda oficial

La distribución manual mediante archivos APK presenta limitaciones importantes en términos de mantenimiento, seguridad, experiencia de usuario y control de versiones. Por este motivo, la publicación en Google Play Store ofrece ventajas fundamentales:

- Instalación sencilla para el alumnado y profesorado.
- Actualización automática de versiones.
- Mayor percepción de fiabilidad y profesionalidad.
- Cumplimiento de estándares de seguridad y privacidad.
- Gestión centralizada de versiones y despliegues progresivos.

Estas características convierten a Google Play en el canal idóneo para la distribución real del sistema en un entorno educativo.

8.5.2 Preparación técnica para la publicación

Antes de proceder a la publicación, será necesario realizar una serie de configuraciones técnicas dentro del proyecto Android generado mediante Capacitor y Android Studio:

- Definición de un **identificador único estable** de la aplicación (*applicationId*).
- Configuración del **versionado semántico** mediante los campos *versionName* y *versionCode*.

- Optimización del tamaño del paquete y compilación en modo *release*.
- Generación y custodia segura de la **clave de firma criptográfica (keystore)**.
- Configuración definitiva de iconos adaptativos, pantallas de carga y recursos gráficos.

Estos pasos garantizan que la aplicación cumpla los requisitos técnicos exigidos por la plataforma y pueda mantenerse de forma sostenible a lo largo del tiempo.

8.5.3 Generación del paquete Android App Bundle (AAB)

Como parte del proceso de publicación, se generará el paquete en formato **Android App Bundle (AAB)**, recomendado oficialmente por Google frente al uso tradicional de APK. Este formato permite una distribución optimizada de los recursos, reduciendo el tamaño final de descarga y adaptando dinámicamente los contenidos a cada dispositivo.

La utilización de AAB facilita además la gestión automática de arquitecturas, resoluciones de pantalla y optimización de recursos, mejorando la eficiencia del despliegue.

8.5.4 Proceso de publicación en Google Play Console

La publicación se realizará mediante la **Google Play Console** [11], siguiendo el flujo estándar establecido por la plataforma:

1. Creación y configuración de la cuenta de desarrollador.
2. Registro inicial de la aplicación.
3. Carga del paquete AAB firmado.
4. Definición de la ficha pública: nombre, descripción, capturas de pantalla, iconos y categoría.
5. Declaración de políticas de privacidad y tratamiento de datos.
6. Configuración de canales de prueba internos y cerrados.
7. Solicitud de revisión y posterior publicación.

Este proceso permite validar el cumplimiento de los estándares de calidad, seguridad y protección de datos exigidos por Google, resultando especialmente relevante en un contexto educativo que implica el tratamiento de datos personales de menores.

8.5.5 Integración en la estrategia de implantación

La publicación en Google Play Store se integrará dentro de la **fase de despliegue parcial** definida en la sección 8.3, una vez validado el funcionamiento general del sistema durante el piloto. De este modo, se garantiza que la distribución oficial se realice sobre una infraestructura validada y con procedimientos operativos consolidados.

8.6 MIGRACIÓN DEL SISTEMA DE ALMACENAMIENTO DE ARCHIVOS

En la versión actual del sistema, los archivos (PDF, MP3 y MP4) se almacenan en formato Base64 dentro de la base de datos, tal y como se describe en el apartado 5.2. Este enfoque resulta adecuado en un entorno académico y de pruebas, ya que simplifica la arquitectura y evita dependencias externas adicionales.

No obstante, en un entorno productivo real presenta limitaciones relevantes en términos de rendimiento, escalabilidad y gestión de recursos, debido al incremento del tamaño de los documentos almacenados y a la sobrecarga derivada de los procesos de codificación y decodificación.

Por este motivo, el plan de implantación contempla una migración progresiva hacia un sistema de almacenamiento externo especializado, como **Firestore Storage** [12] o **Azure Blob Storage** [13], permitiendo desacoplar la gestión de ficheros del núcleo transaccional del sistema.

La migración se ejecutará en cinco fases claramente definidas:

1. Selección del proveedor de almacenamiento.
2. Configuración de variables de entorno e implementación en backend de un servicio de gestión de subida, descarga y eliminación de archivos.
3. Sustitución del modelo Base64 por almacenamiento de metadatos y referencias seguras (URLs).
4. Adaptación del frontend para el consumo directo de dichas referencias.
5. Migración progresiva de los ficheros existentes, priorizando los materiales activos.

Este cambio permitirá mejorar el rendimiento general del sistema, reducir la carga sobre la base de datos y facilitar la escalabilidad futura, sin introducir modificaciones sustanciales en la arquitectura definida en el apartado 3.7. Asimismo, esta migración se llevará a cabo durante la fase de despliegue parcial (Sección 8.3), evitando interferencias en la fase piloto.

Comparativa técnica: Base64 vs almacenamiento externo

Desde el punto de vista de implementación, el cambio implica una simplificación notable en la gestión de archivos. Mientras que el enfoque actual requiere procesos explícitos de codificación y decodificación, el nuevo modelo permite trabajar directamente con referencias a recursos externos.

Modelo actual (Base64 embebido en base de datos):

```
if (this.tarea.materialDeApoyo) {  
  const pdfBlob =  
    this.b64toBlob(this.tarea.materialDeApoyo, 'application/pdf');  
  this.nonSafeUrl = URL.createObjectURL(pdfBlob);  
  this.pdfUrl =  
    this.sanitizer.bypassSecurityTrustResourceUrl(this.nonSafeUrl);  
}
```

Modelo propuesto (almacenamiento externo con referencia):

```
if (this.tarea.materialDeApoyo) {  
    this.pdfUrl = this.sanitizer.bypassSecurityTrustResourceUrl(  
        this.tarea.materialDeApoyo  
    );  
}
```

Este segundo enfoque elimina la necesidad de conversiones intermedias, reduce la carga computacional del servidor y permite delegar la entrega de contenido pesado a infraestructuras especializadas, mejorando significativamente el rendimiento y la escalabilidad del sistema.



Figura 8.2: Logo Firebase Storage

8.7 GESTIÓN PROFESIONAL DEL CORREO ELECTRÓNICO

El correo electrónico constituye un canal crítico de comunicación con los usuarios, ya que se emplea para el envío de credenciales, notificaciones operativas y recordatorios asociados al uso de la plataforma. En un escenario real de implantación, la fiabilidad de este canal impacta directamente en la experiencia de usuario, en la percepción de madurez del sistema y en la seguridad del proceso de alta y recuperación de acceso.

Con el objetivo de garantizar una entrega estable, trazable y alineada con buenas prácticas, se profesionalizará el servicio de correo mediante las siguientes acciones concretas:

1. **Adquisición de un dominio propio:** Se registrará un dominio del centro o del sistema, evitando el uso de dominios genéricos que degradan la confianza del usuario y dificultan la entregabilidad.
2. **Configuración del proveedor de correo:** Se empleará un proveedor transaccional (por ejemplo, Mailjet [12] u otro equivalente), centralizando el envío de correos desde un servicio controlado y monitorizable.
3. **Autenticación del dominio (SPF, DKIM y DMARC):** Se configurarán obligatoriamente estos mecanismos para mejorar la reputación del dominio, reducir el riesgo de suplantación y disminuir la probabilidad de clasificación como spam.
4. **Parametrización segura en el entorno:** Se configurarán el correo emisor, las claves y los endpoints del proveedor mediante variables de entorno, evitando su exposición en el repositorio y facilitando el cambio de proveedor sin afectar al código.

5. **Validación previa y pruebas controladas:** Se realizarán envíos de prueba a proveedores comunes, verificando entregabilidad, cabeceras de autenticación y puntuación anti-spam. Se documentarán los resultados como evidencia de preparación para producción.

Estas tareas se ejecutarán **antes de finalizar la fase piloto** (Sección 8.3) para evitar que el escalado de usuarios se apoye en configuraciones temporales o con reputación insuficiente. Además, este enfoque reduce el impacto de incidencias habituales (correos no entregados o marcados como spam) y facilita la operación diaria del sistema durante el despliegue parcial y completo.

8.8 SEGURIDAD, PROTECCIÓN DE DATOS Y CUMPLIMIENTO NORMATIVO

La seguridad se integra en el plan de implantación como un proceso operativo estructurado, con acciones, revisiones y controles periódicos. Aunque determinadas mejoras técnicas se plantearán como líneas de trabajo futuro, la implantación en un entorno real exige establecer desde el inicio un marco de supervisión y cumplimiento.

8.8.1 Plan operativo de seguridad

Se ejecutará el siguiente plan operativo:

1. **Auditoría inicial previa al piloto** (semana 0–1): revisión de configuración del entorno, variables de entorno, permisos y configuración de base de datos.
2. **Checklist RGPD** (semana 1): revisión de datos tratados, finalidades y criterios de retención.
3. **Configuración de control de accesos** (semana 1): verificación de roles, permisos y validación de rutas según autenticación.
4. **Definición de política de backups y restauración** (semana 2): programación y test básico de recuperación.
5. **Revisiones periódicas** (cada 3 meses): revisión de cuentas activas, logs, accesos y dependencias externas.

8.8.2 Protección de datos personales y marco normativo

En un centro educativo el sistema trata datos personales de alumnado y profesorado, por lo que se operará bajo un marco alineado con RGPD. En la implantación se ejecutarán estas acciones:

1. Identificación de datos personales tratados y su finalidad.
2. Definición de política de retención y borrado (por curso académico, bajas, etc.).
3. Establecimiento de procedimientos para atender derechos de acceso, rectificación y supresión.
4. Documentación interna del tratamiento y responsabilidades del centro (responsable del tratamiento) y del sistema (encargado, si aplica).



8.8.3 Gestión de credenciales y control de accesos

Aunque el sistema ya incorpora autenticación y roles, la implantación exige controles operativos:

1. Revisión mensual de cuentas activas e inactivas.
2. Procedimiento de revocación inmediata ante bajas.
3. Política de sesiones: caducidad, invalidación y revisión de tokens.

8.8.4 Integración de servicios externos (correo, contraseñas, IA)

Las integraciones externas (generación de contraseñas, envío de correo, Groq) se gestionarán con controles concretos:

1. Verificación semanal de disponibilidad del servicio (monitorización básica).
2. Revisión mensual de claves API y caducidad.
3. Revisión trimestral de cambios en políticas o SDKs del proveedor.
4. Definición de contingencias: proveedor alternativo o modo degradado (por ejemplo, desactivar pistas temporalmente).

8.8.5 Protección frente a pérdida de información (backups)

Se ejecutará una política mínima obligatoria:

- Backup incremental diario.
- Backup completo semanal.
- Simulación trimestral de restauración para verificar integridad.

8.8.6 Seguridad en entorno móvil

La exportación Android requiere supervisar elementos específicos:

1. Firma de la aplicación y gestión segura de claves.
2. Validación de permisos solicitados.
3. Verificación de comunicaciones cifradas (HTTPS) y configuración segura de endpoints.

8.9 FORMACIÓN DE USUARIOS Y ADOPCIÓN DEL SISTEMA

La implantación tecnológica se acompaña de un plan de adopción centrado en el factor humano. Aunque en el contexto de este proyecto no se ha llevado a cabo una implantación real completa, se define un programa de formación estructurado que permitiría facilitar la integración progresiva del sistema en un centro educativo real.

Este plan formativo se ejecutaría en paralelo a la implantación técnica, con un calendario definido y una estructura gradual orientada a reducir la resistencia al cambio y favorecer la adopción natural de la herramienta.

8.9.1 Formación del profesorado

La formación del profesorado se estructuraría en tres etapas complementarias:

1. **Sesión inicial (4 horas)** previa al inicio de la fase piloto, centrada en la navegación general del sistema, gestión de grupos, publicación de materiales, creación de tareas y gestión de calificaciones.
2. **Taller práctico** durante la segunda semana del piloto, enfocado en la creación de cuestionarios, revisión de entregas, uso del sistema de mensajería y resolución de dudas operativas.
3. **Sesión de consolidación (2 horas)** tras el primer mes de uso, orientada a la resolución de incidencias recurrentes, revisión de flujos de trabajo y fijación de buenas prácticas.

Este enfoque progresivo permitiría que el profesorado incorpore la herramienta de manera gradual, reduciendo la carga cognitiva y facilitando una transición fluida desde los métodos tradicionales.

8.9.2 Formación del alumnado

La formación del alumnado se plantearía como una introducción eminentemente práctica, apoyándose en la simplicidad de la interfaz descrita en el apartado 6.2. El plan contemplaría:

- Una sesión introductoria de aproximadamente 30 minutos por grupo durante la primera semana de uso.
- Acceso a una guía rápida visual integrada en la propia aplicación.
- Resolución de dudas iniciales durante las dos primeras semanas de utilización.

Este planteamiento permite una rápida adaptación del alumnado sin generar una sobrecarga formativa innecesaria.

8.9.3 Documentación y recursos de apoyo

Como parte del plan de implantación, se prevé la elaboración de un conjunto de recursos de apoyo destinados a facilitar la autonomía de los usuarios y reducir la dependencia de soporte técnico directo:

- Guía rápida (profesorado).
- Guía rápida (alumnado).
- Documento de preguntas frecuentes (FAQ) y guía básica de resolución de incidencias.

Estos materiales se definirían antes del despliegue parcial, priorizando inicialmente una guía mínima y recursos breves de apoyo.

8.10 CRONOGRAMA DE IMPLANTACIÓN

Con el fin de operativizar la estrategia descrita en la Sección 8.3 en un proceso ejecutable y controlable, se define un cronograma orientativo de implantación. Este cronograma organiza las actividades por fases, marca dependencias lógicas (por ejemplo, disponer de infraestructura y configuración mínima antes de iniciar el piloto) y permite asociar hitos verificables para decidir el avance entre etapas.

El cronograma propuesto se adapta a un centro educativo de tamaño medio y contempla una implantación progresiva en tres fases (piloto, parcial y completa), alineada con los tiempos ya establecidos en el propio plan: un piloto de cuatro a seis semanas, un despliegue parcial de dos a tres meses y una transición final hacia despliegue completo. Además, integra tareas transversales (seguridad, correo y formación) para garantizar que el crecimiento de usuarios se realice sobre una base estable y controlada.

La Figura 8.3 muestra el cronograma completo con la secuencia temporal de tareas y los hitos principales: preparación del entorno productivo, arranque del piloto, emisión del informe de validación, ampliación a despliegue parcial, consolidación operativa y paso a despliegue completo. Se incluyen también revisiones periódicas trimestrales asociadas al seguimiento de seguridad y dependencias externas, en coherencia con la Sección 8.8.



Figura 8.3: Cronograma orientativo de implantación

Este cronograma se concibe como un instrumento de control y toma de decisiones: cada fase finaliza con un hito de validación que determina si se avanza a la siguiente etapa o si se ejecuta una iteración correctiva. En este sentido, la validación práctica descrita en la Sección 8.11 puede interpretarse como una evidencia empírica preliminar equivalente a una comprobación propia del piloto, aunque realizada fuera del marco formal completo, reforzando la utilidad de planificar la implantación con un enfoque incremental.

8.11 VALIDACIÓN PRÁCTICA EN ENTORNO EDUCATIVO REAL

Una vez definido el plan de implantación completo descrito en las secciones anteriores, se llevó a cabo una **validación práctica preliminar** del sistema en un entorno educativo real. Aunque esta experiencia no constituye una implantación formal siguiendo todas las fases del plan descrito, sí representa un **primer contacto real con usuarios finales**, aportando un elevado valor académico y permitiendo contrastar la idoneidad funcional, pedagógica y tecnológica de la solución desarrollada.

Esta validación se realizó mediante una sesión de demostración en un centro educativo con el que el autor del proyecto mantiene una vinculación directa como docente, lo que permitió desarrollar la experiencia en un entorno real, familiar y pedagógicamente representativo. Este contexto facilitó un

intercambio cercano y sincero con profesorado y alumnado, aportando una perspectiva especialmente valiosa para la evaluación del sistema.

La sesión contó con la participación de una profesora de lenguaje musical y varios alumnos, con el objetivo de evaluar:

- La facilidad de uso de la aplicación.
- La comprensión de la interfaz por parte del alumnado.
- La adecuación pedagógica de los flujos de trabajo.
- La utilidad real de las funcionalidades incorporadas en este complemento.

Asimismo, esta experiencia permitió detectar pequeños problemas de usabilidad y comportamiento visual que fueron corregidos posteriormente, reforzando la calidad final del sistema y evidenciando la importancia de una fase de validación empírica dentro del ciclo de vida del software educativo.

8.11.1 Contexto de la demostración

La demostración se realizó en la Escuela de Música Manuel Sánchez Alonso, perteneciente a la Asociación Cultural Musical Eladio Guzmán, en colaboración directa con el profesorado del centro y dentro de un contexto de enseñanza habitual del lenguaje musical.

Se presentó la aplicación tanto en su versión web como en su versión móvil Android, con el objetivo de mostrar:

- La coherencia funcional entre plataformas.
- La correcta adaptación visual en dispositivos móviles.
- La utilidad real de las nuevas funcionalidades implementadas.

Durante la sesión, se expusieron las principales áreas de la aplicación: gestión de grupos, publicación de material, tareas, cuestionarios y el nuevo sistema de pistas inteligentes. Los alumnos interactuaron directamente con la plataforma, completando cuestionarios, consultando material y navegando por las distintas ramas de contenido, lo que permitió observar de forma directa su comportamiento y grado de adaptación.

8.11.2 Valoración del profesorado

La profesora participante, Patricia, docente de lenguaje musical en el centro, ofreció una valoración muy positiva del sistema, destacando especialmente su **carácter completo, atractivo e intuitivo**. En particular, subrayó los siguientes aspectos:

- **Facilidad de uso:** La interfaz fue percibida como muy clara y sencilla, permitiendo un manejo natural sin necesidad de formación previa intensiva.

- **Centralización de recursos:** La posibilidad de disponer en un único entorno de materiales, tareas, calificaciones y comunicación fue considerada una mejora sustancial respecto a los métodos tradicionales empleados en el aula.
- **Facilidad de seguimiento:** El acceso inmediato al estado de entregas y calificaciones proporcionó una visión clara y estructurada del progreso del alumnado.
- **Utilidad del sistema de pistas:** Se valoró especialmente la incorporación de ayudas controladas durante la realización de cuestionarios, considerándolas un mecanismo pedagógicamente adecuado para reconducir el razonamiento del alumno sin ofrecer directamente la respuesta.

Asimismo, la docente destacó el **alto potencial de implantación real** de la herramienta en la dinámica habitual del aula, señalando que su uso podría mejorar notablemente la organización del trabajo, la comunicación con el alumnado y la motivación general durante el aprendizaje del lenguaje musical.

8.11.3 Reacciones del alumnado

El alumnado mostró una respuesta muy positiva ante el uso de la plataforma, destacando especialmente:

- La claridad de la interfaz.
- La facilidad de navegación.
- El atractivo visual del entorno.
- La utilidad de las pistas durante los cuestionarios.

Durante la sesión, se observó un alto grado de implicación y curiosidad por parte de los alumnos, que se mostraron especialmente motivados al interactuar con los cuestionarios y los contenidos multimedia. La experiencia resultó llamativa para ellos, favoreciendo una participación activa y una actitud positiva hacia la actividad propuesta.

Además, se comprobó que comprendían rápidamente la lógica de funcionamiento de la aplicación, accedían sin dificultad a los contenidos y realizaban las entregas sin necesidad de explicaciones extensas, lo que refuerza la validez del diseño UX/UI descrito en la sección 6.2.

8.11.4 Problemas detectados y mejoras aplicadas

La validación práctica permitió identificar una serie de pequeños problemas de diseño, usabilidad y comportamiento que no habían sido detectados durante las fases previas de desarrollo y pruebas en entorno controlado. Estos aspectos, aunque no comprometían el funcionamiento general del sistema, sí afectaban a la calidad de la experiencia de usuario y a la percepción de madurez de la aplicación.

Entre los principales puntos detectados se encuentran:

- Ajustes de tamaño en ciertos elementos visuales en pantallas pequeñas.
- Pantalla de carga móvil no funcional.
- Ausencia de iconos representativos y feedback visual insuficiente ante situaciones de error.
- Formularios que no se limpiaban correctamente tras determinadas operaciones.

Estos problemas fueron abordados inmediatamente tras la sesión de validación, dando lugar a una **fase correctiva específica**, orientada a mejorar la estabilidad visual, la claridad de la interfaz y la calidad de la interacción con el usuario. Las correcciones se implementaron de forma iterativa, verificando su impacto directamente en dispositivos reales.

Este proceso pone de manifiesto la importancia de incorporar fases de validación empírica dentro del plan de implantación, ya que permite detectar fricciones reales en el uso cotidiano de la aplicación, imposibles de anticipar completamente durante el desarrollo teórico. Asimismo, evidencia el valor de la observación directa y del feedback inmediato como herramientas fundamentales para la mejora continua del sistema.

8.11.5 Valor académico de la validación empírica

La realización de esta demostración aporta un valor añadido significativo al proyecto desde un punto de vista académico:

- Permite validar empíricamente las decisiones de diseño.
- Refuerza la orientación práctica del trabajo.
- Introduce colaboración externa real.
- Conecta el desarrollo técnico con su impacto pedagógico.

Además, esta experiencia permite afirmar que el sistema no solo es técnicamente viable, sino también funcionalmente adecuado y pedagógicamente coherente con los objetivos educativos planteados.



Figura 8.4: Demostración de la aplicación SolfEdge en un entorno real de aula

8.12 SÍNTESIS DEL PLAN DE IMPLANTACIÓN

El plan de implantación presentado establece una hoja de ruta completa para trasladar la solución desde un entorno académico hacia un contexto real de uso educativo. A través de una estrategia progresiva (Sección 8.3), se asegura que la adopción del sistema se realice de forma controlada, permitiendo validar estabilidad, rendimiento y aceptación por parte de los usuarios antes de su despliegue completo.

Desde el punto de vista técnico, el plan define un proceso de migración hacia un entorno productivo apoyado en servicios escalables (Sección 8.4), garantizando que el crecimiento del número de usuarios pueda abordarse sin reestructurar la arquitectura. Del mismo modo, se contempla la evolución de componentes críticos como el almacenamiento de ficheros (Sección 8.6) y la profesionalización del correo electrónico (Sección 8.7), elementos clave para operar con fiabilidad en escenarios reales.

En paralelo, se integra la seguridad como un proceso operativo de supervisión y cumplimiento (Sección 8.8), y se incorpora un plan de adopción orientado a minimizar fricciones en el uso cotidiano del sistema (Sección 8.9). Aunque el plan no se ha ejecutado en su totalidad durante el desarrollo del proyecto, la validación práctica preliminar realizada (Sección 8.11) aporta evidencia empírica y refuerza la utilidad de estructurar una implantación en fases.

En conjunto, este capítulo completa el ciclo de vida del proyecto más allá del desarrollo funcional descrito en los capítulos 5 y 6, demostrando una visión integral de despliegue, operación y evolución. De este modo, la solución no se presenta únicamente como un prototipo académico, sino como una plataforma con un plan realista de implantación y un potencial de impacto educativo sostenible.

9 CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURO

En este capítulo se presentan las conclusiones finales del proyecto de complemento, evaluando el grado de cumplimiento de los objetivos planteados y el valor añadido aportado respecto al proyecto original, así como una reflexión sobre las principales líneas de trabajo futuro que permitirían seguir ampliando, mejorando y consolidando la solución desarrollada.

9.1 CONCLUSIONES

El objetivo general de este proyecto de complemento ha sido extender e implantar la solución informática desarrollada en el proyecto previo, llevándola a un nivel superior tanto en términos funcionales como de preparación para su uso en entornos educativos reales. A lo largo del desarrollo, este objetivo se ha abordado mediante la mejora progresiva de la aplicación, la incorporación de nuevas funcionalidades, la adaptación a entornos móviles y la definición de un plan de implantación estructurado y realista.

En relación con los subobjetivos definidos, se pueden extraer las siguientes conclusiones:

- **Mejora y refinamiento de funcionalidades existentes:** Se han optimizado diversos aspectos de la aplicación original, mejorando la estabilidad, la usabilidad y la claridad de la interfaz. Estas mejoras han permitido adaptar la solución a flujos de trabajo reales, reforzando su adecuación al contexto educativo y aumentando la calidad de la experiencia de usuario tanto para profesorado como para alumnado.
- **Incorporación de nuevas funcionalidades innovadoras:** El proyecto ha ampliado significativamente el alcance funcional de la plataforma, integrando nuevos mecanismos orientados a enriquecer el proceso de aprendizaje y a reforzar el valor pedagógico de la herramienta. Estas incorporaciones aportan un carácter innovador al sistema y permiten explorar nuevas metodologías educativas apoyadas en tecnología.
- **Adaptación y despliegue en entornos móviles:** Gracias al uso de una arquitectura híbrida y diseño responsive, se ha logrado generar una versión móvil plenamente funcional de la aplicación, garantizando una experiencia de usuario coherente en dispositivos Android. Esta extensión incrementa notablemente la accesibilidad, portabilidad y flexibilidad del sistema, facilitando su integración en la dinámica educativa diaria.
- **Definición de un plan de implantación realista:** Se ha elaborado un plan de implantación integral que abarca aspectos técnicos, organizativos, de seguridad y formativos, estableciendo una hoja de ruta clara para el despliegue progresivo del sistema en centros educativos reales. Este plan

permite transformar un desarrollo académico en una solución con viabilidad operativa, aportando una visión profesional del ciclo de vida del software.

Adicionalmente, la realización de una validación práctica en un entorno educativo real ha permitido contrastar empíricamente la idoneidad funcional, pedagógica y tecnológica del sistema, reforzando la calidad final de la solución y evidenciando la importancia de incorporar procesos de evaluación con usuarios finales.

En conjunto, el proyecto de complemento ha permitido consolidar la plataforma desarrollada, dotándola de una mayor madurez técnica, una funcionalidad ampliada y una proyección realista hacia su implantación en entornos educativos reales, cerrando así de forma coherente el ciclo completo de análisis, desarrollo, validación e implantación.

9.2 LÍNEAS DE TRABAJO FUTURO

A pesar de los avances alcanzados, el sistema mantiene un amplio margen de crecimiento. La propia evolución del complemento ha puesto de manifiesto nuevas necesidades y oportunidades de mejora que configuran un conjunto rico de líneas de trabajo futuro.

9.2.1 Ejecución completa del plan de implantación en un centro educativo

Aunque se ha definido un plan de implantación detallado y se ha realizado una validación preliminar, una de las principales líneas futuras consiste en la **ejecución completa del despliegue en un centro educativo real**, siguiendo todas las fases propuestas.

Este trabajo permitiría:

- Validar el comportamiento del sistema en uso continuado.
- Analizar métricas reales de carga, rendimiento y estabilidad.
- Obtener retroalimentación prolongada del profesorado y alumnado.
- Ajustar procesos organizativos y técnicos en función del uso real.

La implantación completa consolidaría la madurez del sistema y permitiría evaluar su impacto pedagógico de forma longitudinal.

9.2.2 Extensión continua de funcionalidades e innovación pedagógica

El complemento ha incorporado mejoras relevantes, pero la naturaleza del dominio educativo permite una evolución funcional constante. Entre las posibles extensiones futuras destacan:

- Evolución del sistema de ayudas inteligentes mediante mayor personalización, adaptación al progreso del alumno y análisis de errores recurrentes.

- Incorporación de nuevos módulos didácticos interactivos (entrenamiento auditivo, dictados rítmicos, reconocimiento melódico, etc.).
- Desarrollo de herramientas de analítica educativa que faciliten el seguimiento del progreso y la toma de decisiones pedagógicas.

Estas ampliaciones reforzarían el valor diferencial de la plataforma y su alineación con metodologías educativas activas.

9.2.3 Apertura del sistema a usuarios externos y acceso sin autenticación

Tal como se planteaba en el proyecto original, permanece pendiente la apertura parcial del sistema a usuarios no autenticados o no vinculados a centros educativos. Esta línea permitiría:

- Facilitar el acceso a contenidos demostrativos.
- Permitir el registro de usuarios independientes.
- Aumentar la difusión y visibilidad del proyecto.

La implementación de este modelo requeriría un diseño cuidadoso de permisos, privacidad y segmentación funcional.

9.2.4 Tecnologías de mantenimiento, monitorización y gestión de logs

La evolución del sistema hacia escenarios productivos introduce la necesidad de reforzar las capacidades de mantenimiento y supervisión. Como líneas futuras se plantea:

- Integración de sistemas de logging estructurado.
- Centralización de registros y trazabilidad de eventos.
- Incorporación de monitorización, métricas y alertas automáticas.

Estas mejoras permitirían detectar incidencias de forma temprana, reducir tiempos de resolución y aumentar la estabilidad operativa del sistema.

9.2.5 Refuerzo de la seguridad y mejora en la gestión de credenciales

La seguridad emerge como una línea prioritaria de evolución. Entre las mejoras futuras destacan:

- Sustitución del envío directo de credenciales por flujos seguros basados en enlaces de activación y restablecimiento de contraseña.
- Gestión avanzada de secretos mediante servicios dedicados.

- Implementación de políticas de autenticación reforzada y control de accesos.

Estas acciones permitirían elevar significativamente el nivel de protección de datos personales, especialmente relevante en contextos educativos con menores.

En conclusión, el proyecto de complemento no solo ha permitido materializar líneas de evolución previamente planteadas, sino que ha abierto nuevas vías de mejora orientadas a la profesionalización, la seguridad y la sostenibilidad del sistema. De este modo, la solución desarrollada se consolida como una plataforma con un alto potencial de crecimiento, capaz de adaptarse progresivamente a las exigencias técnicas, pedagógicas y organizativas de un entorno educativo real.

BIBLIOGRAFÍA

- [1] Groq Inc., “Groq api documentation – overview.” [Online]. Available: <https://console.groq.com/docs/overview>
- [2] Apache Software Foundation, “Apache cordova documentation.” [Online]. Available: <https://cordova.apache.org/archive/docs/en/12.x/>
- [3] Ionic Team, “Capacitor documentation.” [Online]. Available: <https://capacitorjs.com/docs/v2>
- [4] Google LLC, “Android studio – introduction.” [Online]. Available: <https://developer.android.com/studio/intro?hl=es-419>
- [5] Microsoft Corporation, “Microsoft azure documentation.” [Online]. Available: <https://learn.microsoft.com/es-es/azure/?product=popular>
- [6] —, “Azure app service documentation.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/app-service/>
- [7] —, “Azure key vault documentation.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/key-vault/>
- [8] Docker Inc., “Docker documentation.” [Online]. Available: <https://docs.docker.com/>
- [9] Microsoft Corporation, “Azure container registry documentation.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/container-registry/>
- [10] MongoDB Inc., “Mongodb atlas documentation.” [Online]. Available: <https://www.mongodb.com/docs/atlas/>
- [11] Google LLC, “Google play console documentation.” [Online]. Available: <https://developer.android.com/distribute/console?hl=es-419>
- [12] —, “Firebase storage documentation.” [Online]. Available: <https://firebase.google.com/docs/storage?hl=es-419>
- [13] Microsoft Corporation, “Azure blob storage documentation.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/storage/blobs/>

El complemento del proyecto ha consistido en la ampliación y evolución técnica de la solución informática desarrollada previamente, abordando aspectos avanzados como la arquitectura cloud, el despliegue móvil, la integración de inteligencia artificial y la definición de un plan de implantación orientado a su aplicación en entornos educativos reales.

The complementary part of the project has consisted of the technical extension and evolution of the previously developed software solution, addressing advanced aspects such as cloud architecture, mobile deployment, artificial intelligence integration, and the definition of a deployment plan aimed at its application in real educational environments.

